Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación

CC3069 · Computación Paralela y Distribuida
Ing. Juan Luis García

Proyecto 2

Fuerza bruta sobre DES con Open MPI

INFORME

Esteban Cárcamo	23016
Nicolás Concuá	23197
Diego López	23747

Guatemala, 14 de octubre de 2026

Índice

Introducción 4
Objetivos 4
Qué se hizo 4
Estructura del informe 5

Parte A.1 y A.2 · El algoritmo DES 6
A.1 ¿Qué es DES? 6
A.2 Diagrama de flujo 9
B.1 Verificación del cifrado 10
Referencias 10

Parte A.4 a A.6 · Rutinas de la base y primitivas MPI 11
A.4 Por qué no compila rpc/des_crypt.h y cómo lo reemplazamos 11
A.5 Las rutinas, una por una 11
A.6 Primitivas MPI del aviso de llave 13
Referencias 16

Parte B.1 y B.2 · Pruebas del cifrado y del naive 17
B.1 Cifrar y descifrar desde un .txt 17
B.2 Naive con 4 procesos contra el secuencial 17

Enfoques de paralelización 22
Esqueleto común 22
1. Naive: bloques contiguos (referencia) 23
2. Intercalado (stride) 24
3. Bloque-cíclico (chunks en round-robin) 24
4. Maestro-trabajador (chunks bajo demanda) 25
5. Híbrido MPI + OpenMP (extra) 27
Tabla teórica (56 bits, -np 4, C = 2^16) 28
Lo que gana una alternativa: consistencia, no promedio 29

Análisis de speedup y del comportamiento según la llave 30
1. Lo teórico frente a lo medido 30
2. Por qué el naive es inconsistente 31
3. Por qué las alternativas son consistentes 31
4. El superlineal: aparente, nunca real 32
5. Fugas de eficiencia 32
6. Barrido de procesos, Amdahl y efectividad 33
7. El híbrido: speedup consistente con y sin OpenMP 34
8. De 28 a 56 bits 35

Conclusiones y recomendaciones 36
Conclusiones 36
Retos y cómo se resolvieron con Open MPI 37
Aprendizajes 37

Recomendaciones 37

Referencias 39

Anexo 1 · Catálogo de funciones y librerías 41

des_utils.c · DES sobre OpenSSL 41
options.c · Parámetros 43
problem.c · Problema y reporte 44
cifrar.c · Parte B.1 46
secuencial.c · Parte A.3 47
search_mpi.c · Módulo común MPI 47
Programas MPI 51
hibrido.c · MPI + OpenMP 53
scripts/analizar.py · Análisis de la campaña 55
Librerías 56

Anexo 2 · Bitácora de pruebas y speedups 59

Máquina y software 59
Procedimiento 59
Verificación 60
Pruebas B.1 y B.2 (Nico) 60
Campaña preliminar: tiempos (mediana de 5, en segundos) 60
Speedups (S = T_s / T_p) 61
Barrido de procesos (llave media) 61
Capturas 61

Introducción

El *Data Encryption Standard* (DES) protege un texto con una llave de 56 bits. Hoy esa llave es demasiado corta: probar las 2^56 ≈ 7.2·10^16 combinaciones está al alcance de una máquina dedicada (EFF, 1998), y por eso el NIST retiró el estándar en 2005 (NIST, 2005). Ese mismo ataque, la **fuerza bruta**, es un problema ideal para estudiar la programación paralela: cada llave se prueba de forma independiente, así que el trabajo se reparte casi sin comunicación, y sin embargo la forma de repartirlo decide si el speedup es excelente o nulo.

En este proyecto se cifra un texto con DES y una llave privada, y se recupera la llave probando llaves hasta que el texto descifrado contiene una frase clave conocida de antemano (`"es una prueba de"`). La búsqueda se implementa en C con OpenSSL para DES, Open MPI para el paralelismo entre procesos (MPI Forum, 2023) y, como extra, OpenMP para el paralelismo entre hilos (OpenMP ARB, 2021).

Objetivos

- Implementar y modificar un programa paralelo con MPI en C.
- Descifrar un texto cifrado con DES por fuerza bruta: primero de forma secuencial y después con el reparto *naive* del programa base del curso.
- Proponer y comparar enfoques alternativos de reparto, y medir su speedup, eficiencia, costo y efectividad con las fórmulas del curso (Amdahl, 1967; Gustafson, 1988).
- Explicar por qué el naive tiene un comportamiento inconsistente según la llave, y demostrar que el proyecto **no es superlineal**: los S > p que aparecen son un efecto de búsqueda.

Qué se hizo

Programa	Qué hace	Capítulo
`cifrar`	Cifra y descifra un `.txt` con una llave (Parte B.1)	Parte A y pruebas B
`secuencial`	Fuerza bruta en un solo proceso (Parte A.3); da el T_s	Parte A
`naive`	Bloques contiguos, uno por proceso (Parte B.2)	Pruebas B y enfoques
`intercalado`	El proceso r prueba r, r + p, r + 2p, …	Enfoques
`ciclico`	Chunks de 2^c llaves en round-robin	Enfoques
`maestro_trabajador`	El proceso 0 reparte chunks bajo demanda	Enfoques

Programa	Qué hace	Capítulo
`hibrido`	Intercalado con p procesos MPI y T hilos de OpenMP (extra)	Enfoques

Todos comparten el mismo arranque, el mismo aviso de llave y el mismo reporte (módulo `search_mpi`), y escriben una línea CSV con el mismo formato. Los scripts `verificar.sh`, `campana.sh` y `analizar.py` comprueban que todo halla la llave real, corren la campaña de mediciones (5 repeticiones por caso, mediana) y generan las tablas y gráficas del análisis.

Los extras que se abordaron son **tres o más enfoques alternativos** (se hicieron tres) y **OpenMP**. El clúster en red local se descartó: exige las tres máquinas a la vez, con la misma versión de Open MPI y el mismo sistema operativo, y mezclar macOS con Linux lo volvía frágil para el tiempo disponible.

Estructura del informe

1. **Antecedentes (Parte A):** el algoritmo DES y su diagrama de flujo, las rutinas de la base (`encrypt_des`, `decrypt_des`, `try_key`, `memcpy`, `strstr`) y las primitivas MPI del aviso de llave.
2. **Desarrollo (Parte B):** las pruebas B.1 y B.2 del naive con las llaves del enunciado, los enfoques alternativos con su análisis teórico y el híbrido.
3. **Resultados:** la bitácora de pruebas (Anexo 2) y el análisis de speedup: comparación por llave, barrido de procesos, Amdahl, Gustafson y la discusión del superlineal.
4. **Conclusiones y recomendaciones**, referencias y el catálogo de funciones (Anexo 1).

Parte A.1 y A.2 · El algoritmo DES

A.1 ¿Qué es DES?

El *Data Encryption Standard* (DES) es un algoritmo de cifrado **simétrico por bloques**: la misma llave sirve para cifrar y para descifrar. Fue publicado como estándar federal de Estados Unidos en 1977 (FIPS PUB 46), a partir del cifrado *Lucifer* de IBM, y su última revisión es la FIPS PUB 46-3 de 1999 [1]. El NIST lo retiró en 2005 porque su llave es demasiado corta para la capacidad de cómputo actual [2].

Característica	Valor
Tamaño de bloque	64 bits (8 bytes)
Tamaño de llave	64 bits, de los cuales 8 son de paridad: **56 bits efectivos**
Espacio de llaves	2^56 = 72 057 594 037 927 936 ≈ 7.2 · 10^16
Estructura	Red de Feistel de **16 rondas**
Subllaves	16 subllaves de 48 bits, una por ronda

El último bit de cada uno de los 8 bytes de la llave es de paridad impar y no interviene en el cifrado. Por eso, aunque la llave ocupa 64 bits, la búsqueda por fuerza bruta solo tiene que recorrer 2^56 combinaciones.

Cifrar, paso a paso

1. **Permutación inicial (IP).** Los 64 bits del bloque se reordenan según una tabla fija.
2. **División.** El resultado se parte en dos mitades de 32 bits: L_0 (izquierda) y R_0 (derecha).
3. **16 rondas de Feistel.** Para i = 1, ..., 16:

$$L_i = R_{i-1} \quad R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

Solo la mitad derecha pasa por la función f; la izquierda solo se combina con XOR. 4. **Intercambio final.** Tras la ronda 16 se intercambian las mitades: el bloque queda $R_{16} \| L_{16}$. 5. **Permutación final (IP^{-1}).** Es la inversa de IP y da el bloque cifrado de 64 bits.

La función f

La función f(R, K) recibe 32 bits de la mitad derecha y 48 bits de la subllave [1], [3]:

1. **Expansión E:** de 32 a 48 bits, duplicando 16 de los bits de entrada.
2. **XOR** con la subllave K_i de 48 bits.
3. **Cajas S (S-boxes):** los 48 bits se parten en 8 grupos de 6. Cada caja S_j convierte 6 bits en 4 mediante una tabla de búsqueda: los bits de los extremos eligen la fila y los 4 centrales la columna. Salen 32 bits. Es la única parte **no lineal** de DES y la que le da su seguridad.
4. **Permutación P:** reordena los 32 bits resultantes.

Generación de subllaves

1. **PC-1** (*Permuted Choice 1*): toma la llave de 64 bits, descarta los 8 bits de paridad y reordena los 56 restantes.
2. Los 56 bits se dividen en dos mitades de 28 bits, C_0 y D_0.
3. En cada ronda, C y D se **rotan a la izquierda** 1 posición (rondas 1, 2, 9 y 16) o 2 posiciones (las demás). En total suman 28 posiciones, así que al final vuelven al inicio.
4. **PC-2** (*Permuted Choice 2*): de los 56 bits de C_i ‖ D_i se eligen 48, que forman K_i.

Descifrar

Por la estructura de Feistel, **descifrar es el mismo algoritmo** con las subllaves en orden inverso (K_{16}, K_{15}, …, K_1). La función f no tiene que ser invertible: cada ronda se deshace aplicando el mismo XOR otra vez [3].

Modo ECB y relleno

DES cifra un bloque de 8 bytes. Para textos más largos usamos el modo **ECB** (*Electronic Codebook*): cada bloque se cifra por separado con la misma llave. Nuestro texto, `"Esta es una prueba de proyecto 2"`, mide 32 bytes, es decir, 4 bloques exactos. Si no fuera múltiplo de 8, el último bloque se **rellena con ceros** (función `padded_length`).

ECB **no es seguro** para mensajes reales: dos bloques de texto plano iguales producen dos bloques cifrados iguales, así que se filtran patrones del mensaje. Los modos encadenados como CBC o CTR lo evitan [3]. Para este proyecto no importa, porque lo que se ataca es la llave.

DES ante la fuerza bruta

Con 56 bits efectivos, DES quedó al alcance del *hardware* dedicado a finales de los noventa:

- En 1998, la máquina **Deep Crack** de la Electronic Frontier Foundation (EFF), que costó unos 250 000 dólares, recuperó una llave DES en **56 horas** [4].

- En enero de 1999, el **DES Challenge III**, de la EFF junto con distributed.net, lo logró en **22 horas y 15 minutos** [4].

Nuestro programa secuencial prueba unos **3.9 · 10^6 llaves por segundo** (medido en la Mac de Nico, ver `resultados/secuencial_longitudes.csv`). Recorrer las 2^56 llaves tomaría 7.2 · 10^16 / 3.9 · 10^6 ≈ 1.85 · 10^10 s, unos **585 años** (en promedio, la mitad). Por eso el proyecto paraleliza la búsqueda y, para medir, reduce el espacio con `-b`.

A.2 Diagrama de flujo

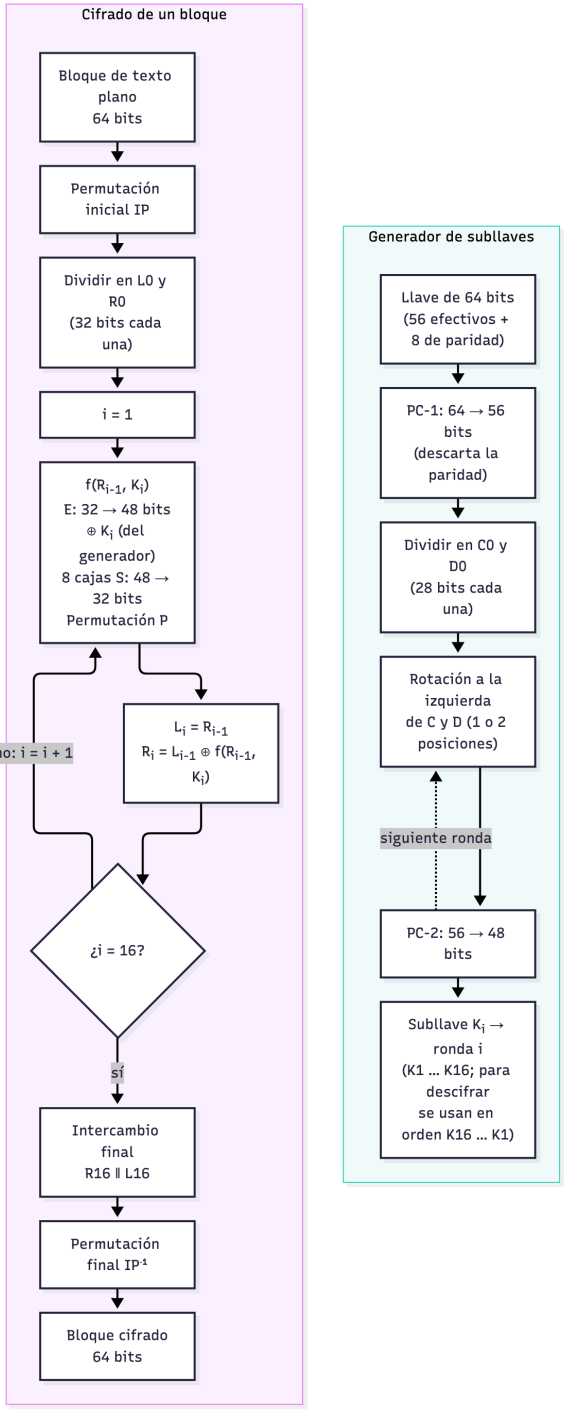
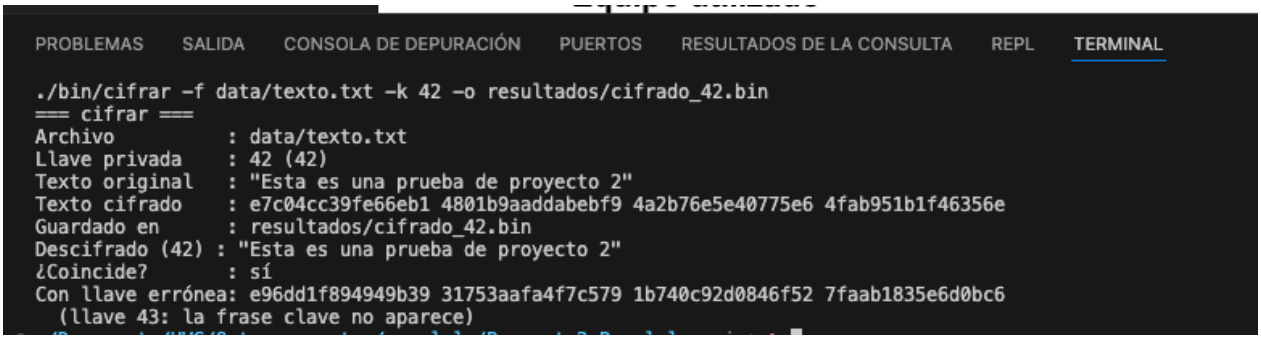


Figura 1. Cifrado de un bloque con DES (izquierda) y generación de las 16 subllaves (derecha). Fuente del diagrama: `docs/img/des-flujo.mmd` (Mermaid).

B.1 Verificación del cifrado

El programa `cifrar` (Parte B.1) cifra el texto con la llave 42 y lo descifra de vuelta:



El cifrado obtenido, `e7c04cc39fe66eb1 4801b9aaddabebf9 4a2b76e5e40775e6 4fab951b1f46356e`, coincide con el de `openssl enc -des-ecb` con la misma llave. Con la llave vecina, 43, sale basura y la frase clave no aparece.

Referencias

[1] National Institute of Standards and Technology, *FIPS PUB 46-3: Data Encryption Standard (DES)*, Gaithersburg, MD, EE. UU., 1999.

[2] National Institute of Standards and Technology, «Announcing Approval of the Withdrawal of Federal Information Processing Standard (FIPS) 46-3, Data Encryption Standard (DES)», *Federal Register*, vol. 70, n.º 96, pp. 28907–28908, 19 de mayo de 2005.

[3] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8.ª ed. Hoboken, NJ, EE. UU.: Pearson, 2020.

[4] Electronic Frontier Foundation, *Cracking DES: Secrets of Encryption Research, Wiretap Politics & Chip Design*. Sebastopol, CA, EE. UU.: O'Reilly, 1998; y EFF, «EFF DES Cracker Machine Brings Honesty to Crypto Debate», comunicado del 19 de enero de 1999.

Parte A.4 a A.6 · Rutinas de la base y primitivas MPI

A.4 Por qué no compila `rpc/des_crypt.h` y cómo lo reemplazamos

El `bruteforce.c` del curso incluye `<rpc/des_crypt.h>` y usa `ecb_crypt` y `des_setparity`. Esas funciones eran parte de la implementación de **Sun RPC** que venía dentro de la biblioteca de C de GNU (glibc). En **glibc 2.32 (2020)** se eliminó por completo Sun RPC, junto con `rpcgen` y sus cabeceras [1], [2]. Lo recomendado desde entonces es la biblioteca externa **libtirpc**, que no viene instalada por defecto.

El motivo fue que esa implementación no soportaba IPv6 y arreglarlo exigía romper la ABI [1]. Por eso, en una distribución moderna (Ubuntu 22.04 o más nuevo, Arch, Fedora) `#include <rpc/des_crypt.h>` falla con «No such file or directory». En macOS tampoco existe.

En lugar de depender de libtirpc, usamos la API DES de **OpenSSL** (`<openssl/des.h>`, enlazada con `-lcrypto`), que está en cualquier sistema y además permite verificar el resultado con la herramienta `openssl enc -des-ecb` [3]. OpenSSL 3 marca esa API como obsoleta porque DES ya no es seguro, pero la mantiene; el aviso se apaga con `-DOPENSSL_SUPPRESS_DEPRECATED`. El módulo `des_utils` reemplaza a las rutinas originales:

`bruteforce.c` (Sun RPC)	**Nuestro proyecto (OpenSSL)**
`encrypt(key, ciph, len)`	`encrypt_des(key, buffer, length)`
`decrypt(key, ciph, len)`	`decrypt_des(key, buffer, length)`
`tryKey(key, ciph, len)`	`try_key(key, cipher, length, phrase, workspace)`
`des_setparity(k)`	`DES_set_odd_parity(&k)`
`ecb_crypt(k, buf, len, DES_ENCRYPT)`	`DES_set_key_unchecked` + `DES_ecb_encrypt` por bloque

A.5 Las rutinas, una por una

`encrypt_des` y `decrypt_des`

Ambas llaman a la misma función interna, `run_des`, y solo cambian la dirección (`DES_ENCRYPT` o `DES_DECRYPT`). Trabajan *in-place*: el resultado sobrescribe el buffer.

1. **Armar la llave (`build_des_key`).** La llave es un número de 56 bits, pero DES pide 8 bytes en los que el bit menos significativo de cada uno es de paridad. Igual que el `decrypt()` original, los 56 bits se reparten en grupos de 7: el byte i guarda los bits $7i$ a $7i+6$ de la llave en sus posiciones 1 a 7, y `DES_set_odd_parity` llena la posición 0 para que cada byte tenga paridad impar.
2. **Agenda de subllaves.** `DES_set_key_unchecked` calcula las 16 subllaves de 48 bits (PC-1, rotaciones y PC-2, ver la Parte A.1). Usamos la versión *unchecked* porque la normal rechaza las llaves débiles de DES, y en una búsqueda por fuerza bruta hay que probarlas todas.
3. **ECB bloque a bloque.** `DES_ecb_encrypt` procesa cada bloque de 8 bytes por separado con la misma agenda. La longitud siempre es múltiplo de 8 gracias al relleno con ceros.

`try_key`

Es el corazón de la búsqueda: prueba una llave candidata.

1. Copia el cifrado al buffer `workspace` con `memcpy`.
2. Le pone un `'\0'` al final para poder tratarlo como cadena de C.
3. Lo descifra con la llave candidata (`decrypt_des`).
4. Busca la frase clave con `strstr`. Si aparece, la llave es la correcta (o, con muy poca probabilidad, un falso positivo, que el reporte marca).

El `workspace` lo reserva **una sola vez** quien llama, en lugar de pedir memoria en cada intento como hacía el original. Así se evitan millones de `malloc` y `free`.

`memcpy`

`memcpy(destino, origen, n)` copia `n` bytes de un buffer a otro. Aquí es indispensable porque `decrypt_des` descifra *in-place*: si descifráramos directamente el cifrado original, el primer intento fallido lo destruiría y ya no habría nada que probar con la siguiente llave. Con la copia, el cifrado original queda intacto durante toda la búsqueda [4].

`strstr`

`strstr(texto, frase)` devuelve un puntero a la primera aparición de `frase` dentro de `texto`, o `NULL` si no aparece [4]. Es la prueba de que la llave es correcta: con una llave errónea, el descifrado es basura y la frase no aparece. Por eso el texto debe terminar en `'\0'` y por eso `problem_prepare` rechaza de entrada una frase que no esté en el texto original.

Un intento de llave, en secuencia

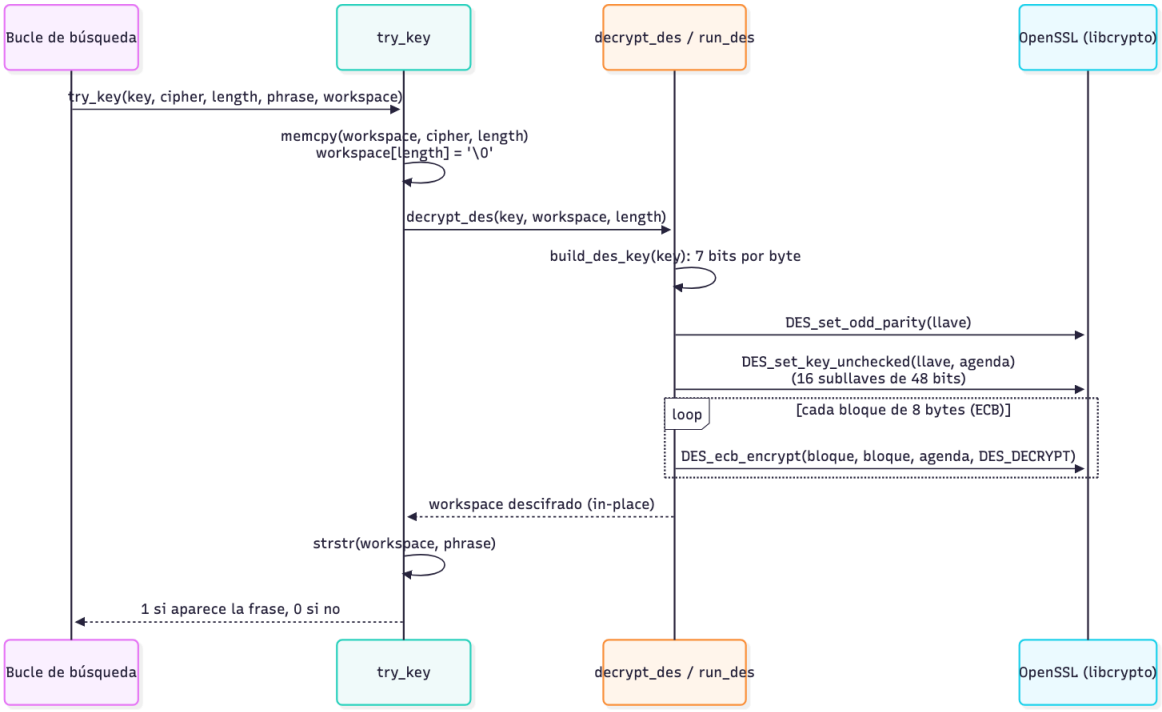


Figura 2. Lo que ocurre en cada llamada a `try_key`.

A.6 Primitivas MPI del aviso de llave

Primitiva	Tipo	Para qué la usamos
`MPI_Irecv`	No bloqueante	Dejar puesta una «antena» que reciba el aviso de llave encontrada
`MPI_Send`	Bloqueante	Que quien halla la llave avise a todos los procesos
`MPI_Wait`	Bloqueante	Cerrar la recepción pendiente antes de terminar
`MPI_Test`	No bloqueante	Revisar, sin detenerse, si ya llegó el aviso

`MPI_Irecv`

`MPI_Irecv(buf, count, datatype, source, tag, comm, &request)` **inicia** una recepción y regresa de inmediato, sin esperar a que llegue el mensaje [5]. Devuelve un `MPI_Request` con el que después se consulta o se espera. En el naive, cada proceso, antes de buscar, publica:

```
MPI_Irecv(found_msg, 3, MPI_UINT64_T, MPI_ANY_SOURCE, FOUND_TAG, MPI_COMM_WORLD,
&request);
```

`MPI_ANY_SOURCE` permite recibir el aviso venga de quien venga, y `found_msg` guarda `{llave, proceso que la halló, sus iteraciones}`. Mientras tanto, el proceso sigue probando llaves: por eso es la «antena».

`MPI_Send`

`MPI_Send(buf, count, datatype, dest, tag, comm)` envía un mensaje y no regresa hasta que el buffer se puede reutilizar [5]. El proceso que encuentra la llave envía el aviso a **todos**, **incluido él mismo**, para que su propio `MPI_Irecv` también se complete y todos cierren igual. Como el mensaje es de solo 24 bytes y cada destino ya tiene su recepción publicada, el envío termina casi de inmediato.

`MPI_Wait`

`MPI_Wait(&request, &status)` bloquea hasta que la operación no bloqueante asociada termina [5]. Al salir del bucle, todos los procesos hacen `MPI_Wait` sobre su antena: así garantizan que `found_msg` ya tiene el aviso antes de usarlo y que no queda ninguna operación pendiente al llamar a `MPI_Finalize`. Si el proceso agotó su bloque sin hallar nada, `MPI_Wait` lo deja esperando hasta que otro avise.

¿Por qué hace falta `MPI_Test`?

El `bruteforce.c` original revisa una variable `found` en cada vuelta, pero esa variable solo cambia si la biblioteca MPI **avanza** la recepción. Open MPI, por defecto, no tiene un hilo que procese mensajes en segundo plano: el progreso de las comunicaciones solo ocurre **dentro de llamadas a MPI** [5]. Un proceso que solo ejecuta `try_key` nunca entra a MPI, así que el aviso se queda en el sistema sin entregarse y el proceso termina su bloque completo.

Por eso, cada `POLL_INTERVAL` = 4096 llaves, el naive llama `MPI_Test(&request, &done, MPI_STATUS_IGNORE)`: le da a MPI la oportunidad de avanzar y, si el aviso ya llegó, `done` vale 1 y el proceso sale del bucle. Probar 4096 llaves toma alrededor de 1 ms, así que el costo de la revisión es despreciable y el retraso máximo para enterarse es de ese orden.

Flujo de mensajes del naive con 4 procesos

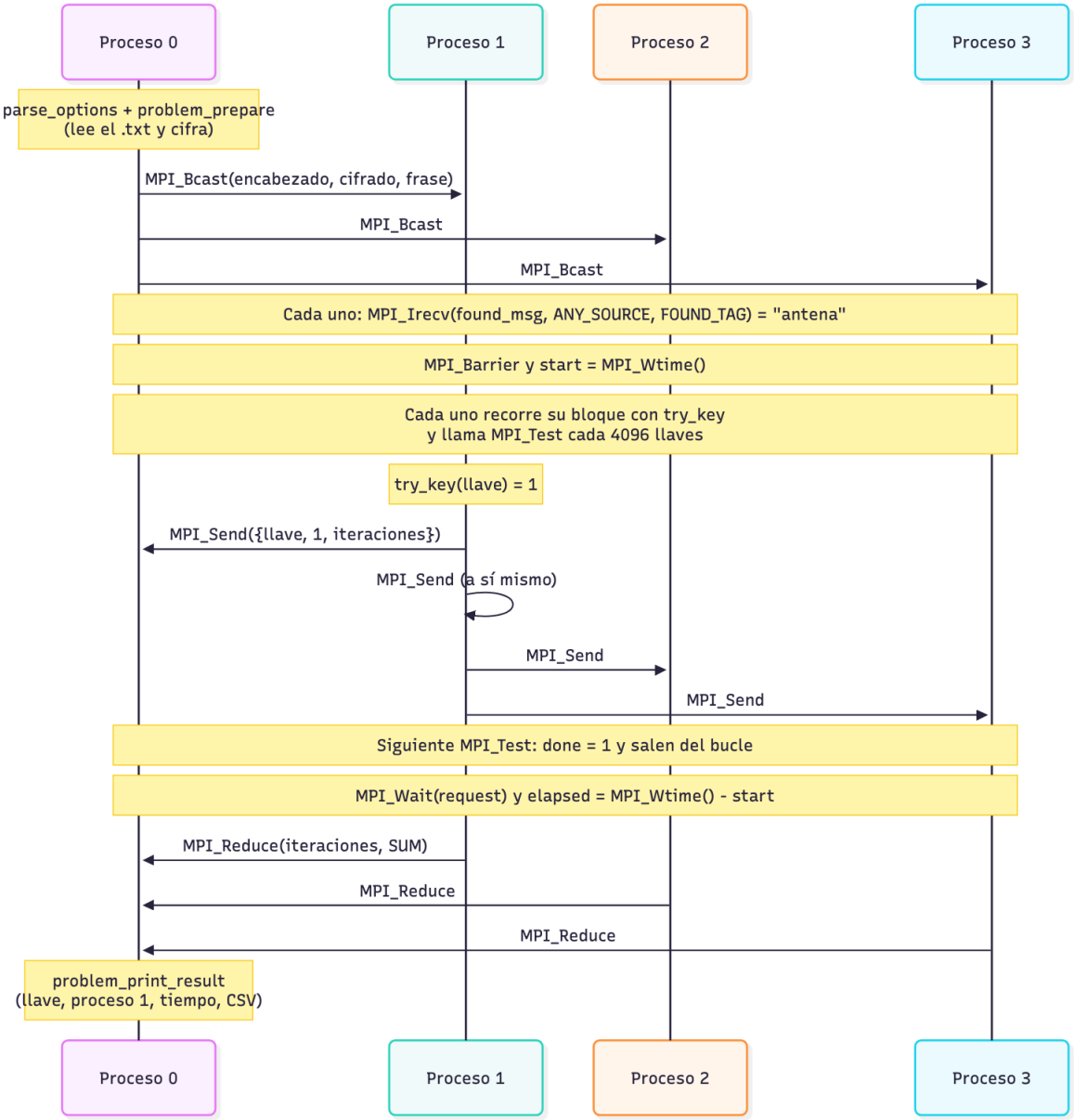


Figura 3. El proceso 1 encuentra la llave: envía el aviso a los 4 procesos, cada uno lo detecta en su siguiente `MPI_Test`, cierra con `MPI_Wait` y el proceso 0 reúne las iteraciones con `MPI_Reduce` y reporta el resultado.

Además del aviso, el naive usa `MPI_Bcast` para repartir el problema (solo el proceso 0 lee el archivo y cifra), `MPI_Barrier` para que todos arranquen el reloj (`MPI_Wtime`) a la vez y `MPI_Reduce` para sumar las llaves probadas. Cuando hay límite de tiempo (`-t`), el proceso 0 envía el mismo aviso con la llave imposible `UINT64_MAX`, que el reporte traduce a «no encontrada».

Referencias

[1] Fedora Project, «Changes/SunRPCRemoval: Removal of Sun RPC Interfaces From glibc», *Fedora Project Wiki*. Disponible: https://fedoraproject.org/wiki/Changes/SunRPCRemoval

[2] F. Weimer, «[PATCH] Remove --enable-obsolete-rpc configure flag», lista de correo *libc-alpha*, julio de 2020. Disponible: https://inbox.sourceware.org/libc-alpha/20200711181508.GA9224@x230/t

[3] OpenSSL Project, «DES_random_key, DES_set_key, DES_ecb_encrypt…: DES encryption», *OpenSSL 3 Manual Pages*. Disponible: https://docs.openssl.org/3.0/man3/DES_random_key/

[4] ISO/IEC 9899:2011, *Programming languages — C*, secciones 7.24.2.1 (`memcpy`) y 7.24.5.7 (`strstr`), 2011.

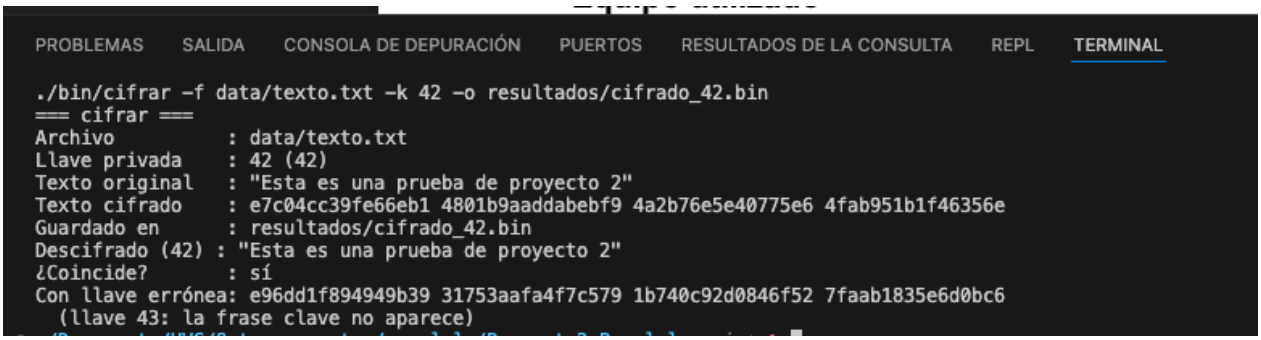
[5] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, versión 4.1, noviembre de 2023, capítulo 3: comunicación punto a punto (`MPI_Send`) y comunicación no bloqueante (`MPI_Irecv`, `MPI_Wait`, `MPI_Test`).

DESARROLLO

Parte B.1 y B.2 · Pruebas del cifrado y del naive

Todas las corridas de esta sección se hicieron en la MacBook Pro de Nico (Apple M4 Pro con 8 núcleos de rendimiento y 4 de eficiencia, macOS 26.6.1, Open MPI 5.0.11 y OpenSSL 3 de Homebrew), con el texto `data/texto.txt` y la frase clave `"es una prueba de"`. Los datos crudos están en `resultados/pruebas_b2.csv`; el script que los genera es `scripts/pruebas_b2.sh`.

B.1 Cifrar y descifrar desde un `.txt`



El programa `cifrar` carga el texto de `data/texto.txt`, lo cifra con la llave 42 que recibe por parámetro (`-k 42`) y lo descifra de vuelta. El texto descifrado coincide con el original, y el cifrado (`e7c04cc39fe66eb1 4801b9aaddabebf9 4a2b76e5e40775e6 4fab951b1f46356e`) es el mismo que produce `openssl enc -des-ecb` con esa llave. Como contraejemplo, con la llave vecina (43) sale basura y la frase clave no aparece: eso es justo lo que la búsqueda usa para distinguir la llave correcta.

B.2 Naive con 4 procesos contra el secuencial

Llaves del enunciado, con N = 2^56:

Prueba	Llave	Posición
a	123456	Muy al inicio del espacio, dentro del bloque del proceso 0
b	N/4 − 1 = 18014398509481983	**Última** llave del bloque del proceso 0
c	N/4 = 18014398509481984	**Primera** llave del bloque del proceso 1

Con `-np 4`, el naive asigna a cada proceso un bloque contiguo de N/4 = 2^54 llaves: el proceso 0 prueba de 0 a N/4 − 1, el proceso 1 de N/4 a N/2 − 1, y así sucesivamente.

Resultados con 56 bits (límite de 120 s)

Prueba	Programa	¿La halló?	Proceso	Iteraciones	Llaves probadas	Tiempo
a	secuencial	sí	0	123 457	123 457	0.054 s
a	naive -np 4	sí	0	123 457	492 097	0.034 s
b	secuencial	no (límite)	—	—	459 493 376	120.0 s
b	naive -np 4	no (límite)	—	—	1 689 812 992	120.0 s
c	secuencial	no (límite)	—	—	433 438 720	120.0 s
c	naive -np 4	**sí**	**1**	**1**	12 289	**0.0012 s**

La prueba **a** es tan corta que la primera corrida tiene ruido de arranque. Repetida 5 veces, la mediana es 0.032 s para el secuencial y 0.034 s para el naive.

Con 56 bits, las pruebas b y c no terminan en el secuencial, así que su tiempo se **estima** con el ritmo medido en esa misma corrida de 120 s, r_s ≈ 3.83 · 10^6 llaves/s:

- T_s(b) ≈ (llave + 1) / r_s = 2^54 / 3.83 · 10^6 ≈ 4.70 · 10^9 s ≈ **149 años**.
- T_s(c) ≈ (2^54 + 1) / r_s ≈ 4.70 · 10^9 s ≈ **149 años**, prácticamente lo mismo.

En el naive, la llave b es la última del bloque del proceso 0, que la alcanza tras 2^54 intentos. Cada proceso probó unas 3.52 · 10^6 llaves/s (1 689 812 992 / 4 / 120 s), así que T_p(b) ≈ 2^54 / 3.52 · 10^6 ≈ 5.1 · 10^9 s ≈ **162 años**.

Resultados con 28 bits (b y c sí terminan)

Con `-b 28` los presets escalan (b = 2^26 − 1, c = 2^26) y conservan su posición relativa: b sigue siendo la última llave del proceso 0 y c la primera del proceso 1.

Prueba	Programa	Proceso	Iteraciones	Tiempo
b	secuencial	0	67 108 864	18.20 s
b	naive -np 4	0	67 108 864	19.00 s
c	secuencial	0	67 108 865	18.56 s
c	naive -np 4	1	1	0.0012 s

Speedup

S = T_s / T_p. Además del speedup de tiempo reportamos el **speedup de rendimiento**, S_rend = (llaves/s del naive) / (llaves/s del secuencial), que mide cuánto trabajo útil hacen los 4 procesos juntos sin importar dónde cae la llave.

Prueba	T_s	T_p (naive - np 4)	S = T_s / T_p	Comentario
a · 56 bits	0.032 s	0.034 s	≈ 0.9	El proceso 0 hace exactamente el mismo trabajo que el secuencial
b · 56 bits	≈ 149 años (est.)	≈ 162 años (est.)	≈ 0.9	Ninguna ganancia
c · 56 bits	≈ 149 años (est.)	0.0012 s	≈ 3.9 · 10^12	**Aparente**: efecto de búsqueda
b · 28 bits	18.20 s	19.00 s	0.96	Ninguna ganancia, medido completo
c · 28 bits	18.56 s	0.0012 s	≈ 1.6 · 10^4	**Aparente**: efecto de búsqueda

En la prueba b, con 56 bits, los 4 procesos probaron juntos 14.1 · 10^6 llaves/s contra 3.83 · 10^6 del secuencial: **S_rend ≈ 3.7 ≤ p = 4**. El naive sí trabaja cuatro veces más rápido; lo que pasa es que en b todo ese trabajo extra se hace en bloques donde la llave no está.

d) Reflexión

El tiempo del naive no depende de cuántas llaves hay que probar en total, sino de **dónde cae la llave dentro del bloque del proceso que la tiene**: T_p ≈ (llave − inicio de su bloque + 1) / ritmo de un proceso.

- **Prueba a (123456):** cae al inicio del bloque del proceso 0. El proceso 0 hace exactamente los mismos 123 457 intentos que el secuencial, y los otros tres procesos no aportan nada. S ≈ 1: paralelizar no ayudó, solo agregó el costo de arrancar MPI.
- **Prueba b (N/4 − 1):** es la **última** llave del proceso 0. Ese proceso tiene que recorrer sus 2^54 llaves, unos 162 años, igual que el secuencial (149 años). Los otros tres procesos terminan sus bloques sin encontrar nada. S ≈ 1, o un poco menos por el costo de MPI_Test.
- **Prueba c (N/4):** es la **primera** llave del proceso 1. La encuentra en **un solo intento**, en un milisegundo, mientras que el secuencial tardaría 149 años. S ≈ 4 · 10^12, muchísimo mayor que p = 4.

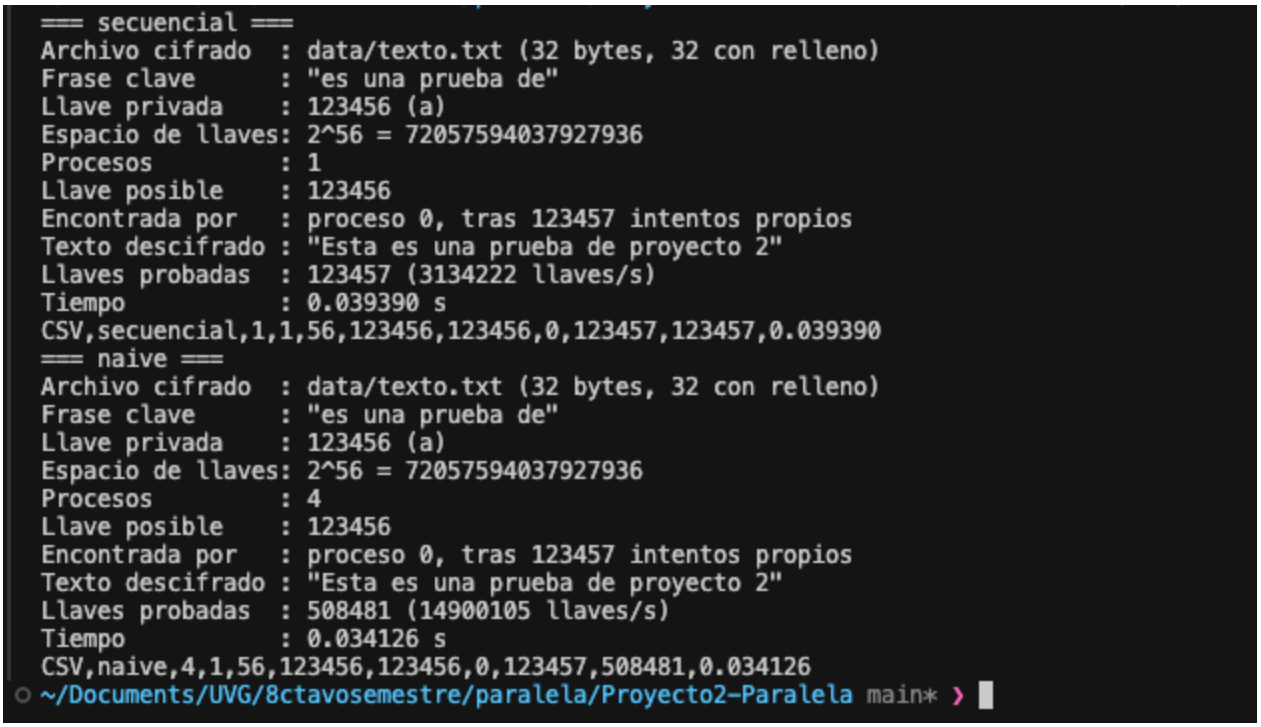
Las llaves b y c son **vecinas** (difieren en 1) y, sin embargo, una da el peor resultado posible y la otra un speedup astronómico. Ese S > p de la llave c **no es superlinealidad real**: es el *efecto de búsqueda*. El naive no hizo el trabajo más rápido; simplemente empezó a buscar justo donde estaba la llave. El speedup de rendimiento, que nunca pasa de p (≈ 3.7 en nuestras corridas), muestra lo que el paralelismo realmente aporta.

Esto deja ver la debilidad del reparto en bloques contiguos: el resultado depende de la suerte, no del algoritmo. Da una «falsa confianza» si se prueba con llaves que caen justo al inicio de un bloque, y no ayuda nada si la llave cae al final del primero. Los enfoques alternativos de la Fase 2 (intercalado, bloque-cíclico y maestro-trabajador) reparten las llaves de forma que todos los procesos avancen a la vez por las zonas bajas del espacio, y así el speedup se vuelve consistente, cercano a p para cualquier llave.

Capturas

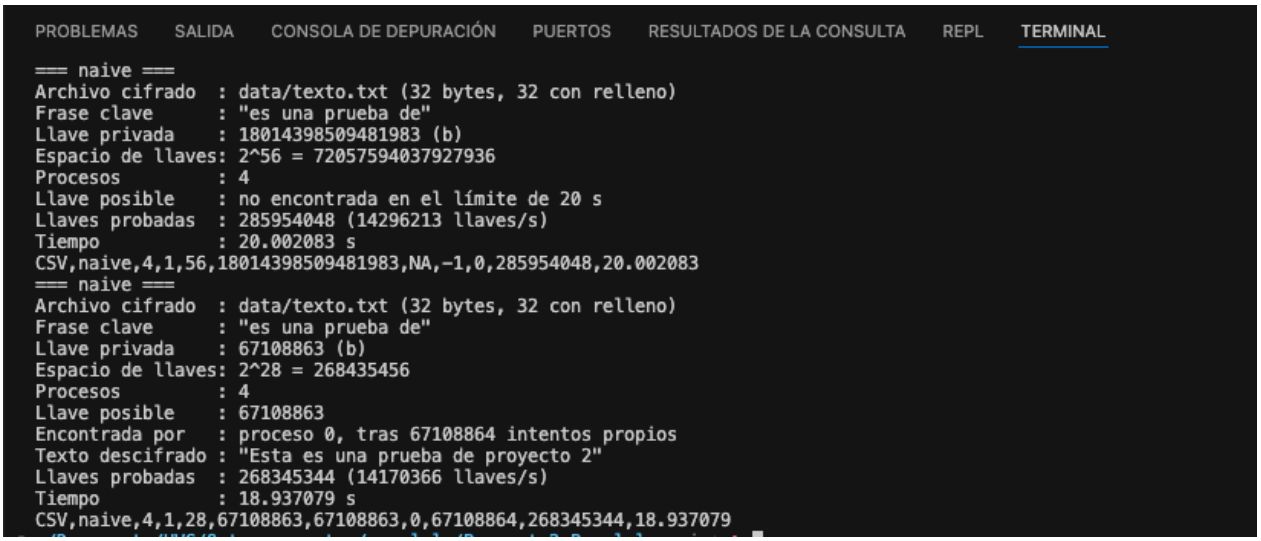
Para las capturas, las corridas que no terminan usan un límite de 20 s en lugar de 120 s; las tablas de arriba salen de la corrida completa de `scripts/pruebas_b2.sh`.

Prueba a



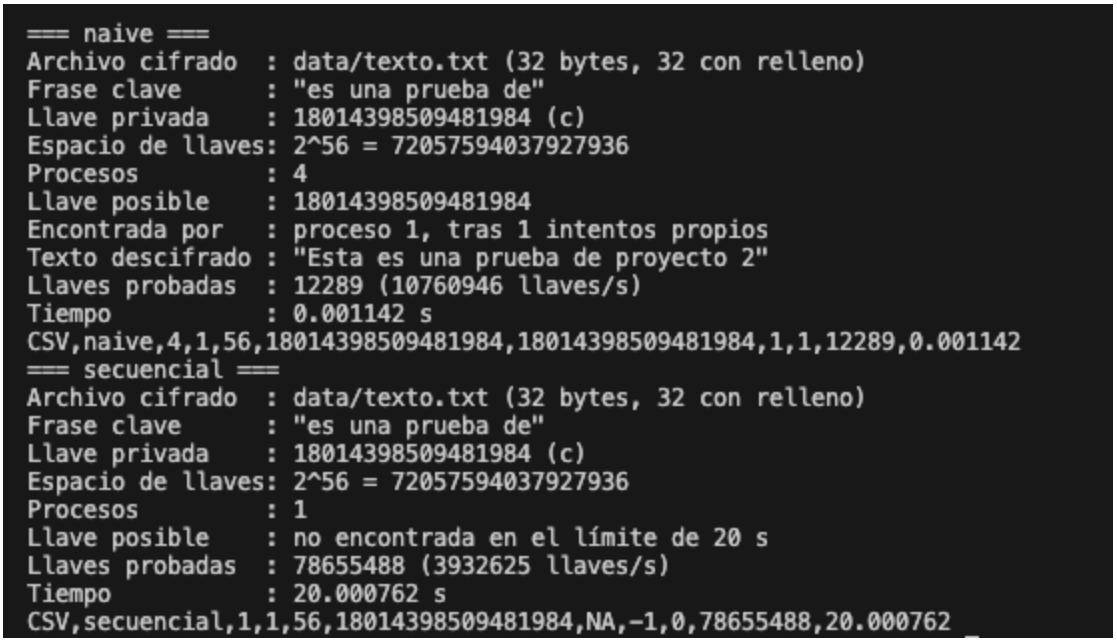
Secuencial y naive con la llave 123456: los dos la hallan en el proceso 0 tras 123 457 intentos.

Prueba b



Naive con la llave b: con 56 bits no la halla en el límite; con 28 bits la halla el proceso 0 tras recorrer su bloque completo (2^26 intentos, 18.9 s).

Prueba c



Naive con la llave c: el proceso 1 la halla al primer intento (0.001 s); el secuencial no la halla en el límite.

Enfoques de paralelización

Los cinco programas MPI (y el híbrido, que añade OpenMP) resuelven el mismo problema con el mismo arranque, el mismo aviso de llave y el mismo reporte (módulo `search_mpi`). Lo único que cambia entre ellos es **qué llaves prueba cada proceso y en qué orden**. Este capítulo describe cada reparto, predice con una fórmula quién halla la llave y en cuántas iteraciones, y discute qué gana y qué pierde cada uno.

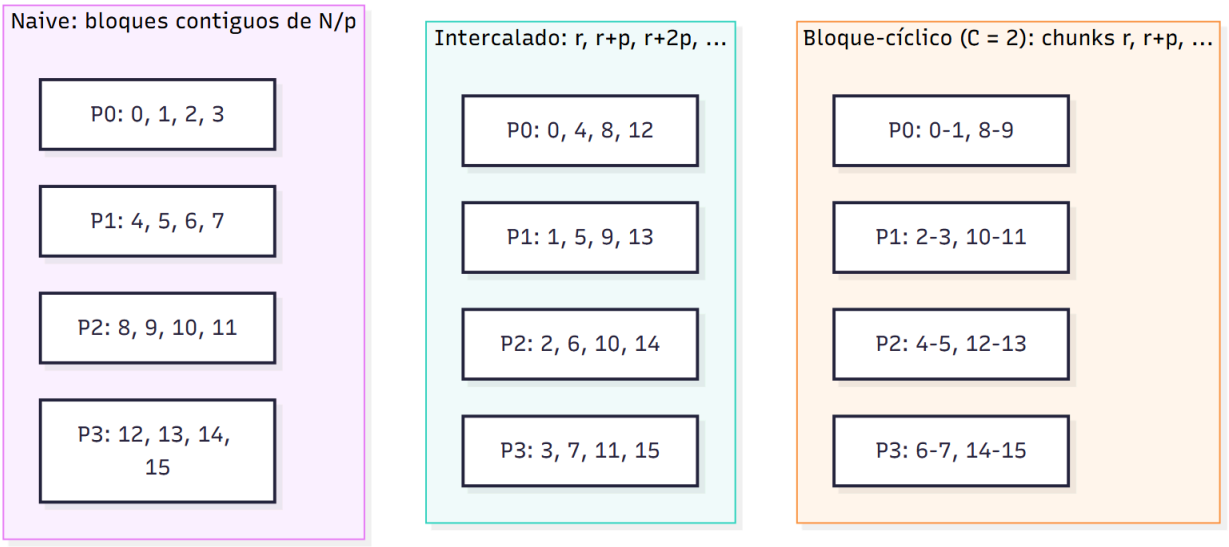
Esqueleto común

```
proceso 0: lee parámetros y archivo, cifra el texto
MPI_Bcast(encabezado, cifrado, frase)            # errores: salen una vez y todos cierran
MPI_Irecv(aviso)                                 # la "antena", puesta antes de buscar
MPI_Barrier; inicio = MPI_Wtime()
para cada llave k del reparto de este proceso:
    cada POLL_INTERVAL = 4096 llaves: MPI_Test(antena); si llegó el aviso, salir
    si try_key(k): MPI_Send(aviso {k, rango, iteraciones}) a todos; salir
MPI_Wait(antena); T_p = MPI_Wtime() - inicio
MPI_Reduce(llaves probadas, SUMA) y el proceso 0 reporta
```

El sondeo cada 4096 llaves es necesario porque Open MPI no hace avanzar la recepción mientras el proceso calcula; su costo es despreciable (una llamada por cada 4096 descifrados). El aviso se envía también a uno mismo, para que su propio `MPI_Irecv` se complete y el `MPI_Wait` final no quede esperando.

Notación. N = 2^bits llaves, p procesos, k la llave buscada. Las **iteraciones** son las llaves que probó el proceso que la halló. El secuencial prueba k + 1 llaves, así que el speedup que predice cada reparto, sin máquina de por medio, es

> S_iter = (k + 1) / iteraciones del que la halla.



1. Naive: bloques contiguos (referencia)

Idea. El espacio se parte en p bloques de N/p llaves; el proceso r recorre el suyo de principio a fin. Es el reparto del `bruteforce.c` del curso.

```
bloque = N / p
inicio = r · bloque;  fin = (r == p - 1) ? N : inicio + bloque
para k = inicio .. fin - 1: probar k
```

Quién la halla. El proceso $r = \lfloor k / (N/p) \rfloor$, tras $k - r \cdot N/p + 1$ iteraciones. Entonces

> $S_{iter} = (k + 1) / (k - r \cdot N/p + 1)$,

que va de **1** (la llave cae en el bloque del proceso 0, que hace el mismo trabajo que el secuencial) a **≈ N/p** (la llave es la primera de un bloque que no es el primero).

- **Ventajas:** el más simple; recorre llaves contiguas; no necesita comunicación salvo el aviso.
- **Desventajas:** el speedup depende por completo de dónde cae la llave. Con llaves pequeñas (las que más probablemente use un humano) no gana nada.
- **Fuga que domina: desbalance η_b** respecto a la llave: los $p - 1$ procesos que no tienen la llave trabajan en vano. Para una llave al azar, el que la tiene recorre en promedio la mitad de su bloque.

2. Intercalado (stride)

Idea. El proceso r prueba r, r + p, r + 2p, …: todos avanzan a la vez por las llaves bajas y por las altas.

```
para k = r, r + p, r + 2p, ... < N: probar k
```

Quién la halla. El proceso k mod p, tras ⌊k/p⌋ + 1 iteraciones:

> S_iter = (k + 1) / (⌊k/p⌋ + 1) ≈ p, y **nunca mayor que p**.

- **Ventajas:** speedup ≈ p para **cualquier** llave; el mismo código que el naive cambiando dos líneas; sin comunicación extra.
- **Desventajas:** cada proceso salta de p en p, así que no aprovecha llaves contiguas (aquí no importa: cada llave se prueba con su propio *key schedule*). Es un reparto estático: si un núcleo es más lento, todos lo esperan.
- **Fuga que domina: comunicación η_c** mínima (el sondeo y el aviso) y la que pone el hardware: con más núcleos activos baja el turbo, y el i9 y el Apple Silicon mezclan núcleos de rendimiento y de eficiencia.

3. Bloque-cíclico (chunks en round-robin)

Idea. El espacio se parte en chunks de C = 2^c llaves (`-c`, por defecto 16). El proceso r recorre los chunks r, r + p, r + 2p, … y dentro de cada uno avanza en orden. Es el punto medio entre los dos anteriores: con C = N/p es el naive y con C = 1 es el intercalado.

```
para j = r, r + p, r + 2p, ... < N / C:
    para k = j·C .. (j + 1)·C - 1: probar k
```

Quién la halla. Con j = ⌊k/C⌋, el proceso j mod p, tras ⌊j/p⌋·C + (k mod C) + 1 iteraciones:

> S_iter = (k + 1) / (⌊j/p⌋·C + (k mod C) + 1) ≈ p cuando k ≫ p·C.

Para llaves diminutas (k < p·C) el comportamiento se parece al del naive: la llave 123456 con C = 2^16 cae en el chunk 1, del proceso 1, en su posición 57 920, y da S_iter ≈ 2.13.

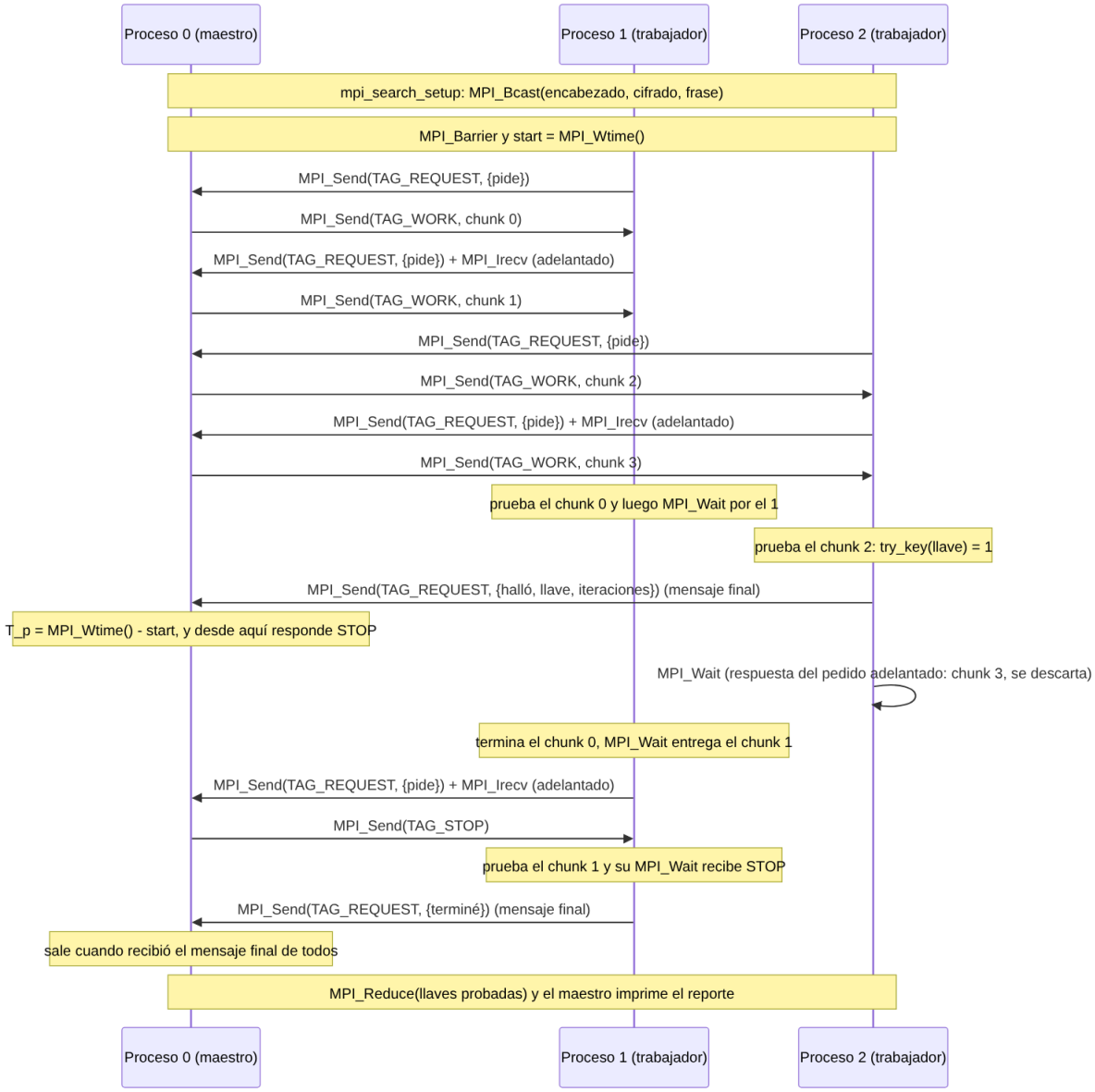
- **Ventajas:** reparto parejo como el intercalado, recorrido contiguo como el naive; el tamaño de chunk permite moverse entre los dos.
- **Desventajas:** un parámetro más; con chunks grandes respecto a la llave se pierde la consistencia.
- **Fuga que domina: desbalance η_b** dentro del último chunk (como mucho C llaves) y la misma comunicación mínima del intercalado.

4. Maestro-trabajador (chunks bajo demanda)

Idea. El proceso 0 es el **maestro** y no busca: reparte chunks de C llaves a quien los pide. Los procesos 1..p−1 son **trabajadores**: piden un chunk, lo prueban y piden otro. Es el único reparto **dinámico**: un núcleo más lento simplemente pide menos chunks.

```
maestro:                                  trabajador:
  activos = p - 1                           pedir; recibir respuesta
  mientras activos > 0:                     mientras la respuesta sea WORK:
    recibir pedido (MPI_ANY_SOURCE)           pedir el siguiente (MPI_Send + MPI_Irecv)
    si es mensaje final (FOUND o DONE):       probar el chunk actual
      si FOUND: T_p = ahora; parar = 1          si halló k: enviar FOUND {k, iter}
      activos -= 1                                        MPI_Wait(respuesta) y terminar
    si no: responder STOP si parar o no       MPI_Wait(respuesta adelantada)
           quedan chunks; si no, WORK         enviar DONE
```

Protocolo. Dos invariantes evitan cuelgues: cada pedido recibe **exactamente una** respuesta, y cada trabajador termina con **un único mensaje final** (FOUND o DONE) que no espera respuesta. Como MPI no adelanta mensajes del mismo origen con la misma etiqueta, cuando el maestro lee el mensaje final de un trabajador ya atendió todos sus pedidos anteriores. El trabajador pide el chunk siguiente **antes** de probar el actual (doble buffer con `MPI_Send` + `MPI_Irecv` + `MPI_Wait`), así la ida y vuelta con el maestro se oculta detrás del cálculo.



Quién la halla. Con $W = p - 1$ trabajadores que van al mismo ritmo, los chunks se reparten casi en round-robin entre ellos, así que se comporta como el bloque-cíclico con W procesos:

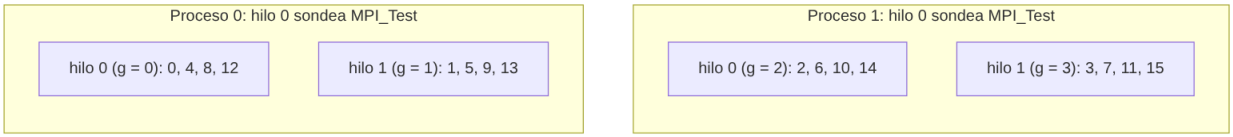
> $S_iter \approx p - 1$ (≈ 3 con `-np 4`), y la eficiencia ideal es $E = (p - 1)/p$ (0.75 con `-np 4`).

- **Ventajas:** balanceo automático con núcleos desiguales o máquinas compartidas; respeta `-t` con resolución de un chunk.

- **Desventajas:** el maestro no busca, así que con pocos procesos pierde 1/p del poder de cómputo; con `-np 2` hay un solo trabajador y no puede ganarle al secuencial.
- **Fuga que domina: parte secuencial η_f** estructural (el maestro, que solo reparte) y **comunicación η_c** (un pedido y una respuesta por chunk, ocultos en parte por el pedido adelantado). Con más procesos, $(p - 1)/p \to 1$.

5. Híbrido MPI + OpenMP (extra)

Idea. Es el intercalado con dos niveles: p procesos MPI y T hilos de OpenMP por proceso (`OMP_NUM_THREADS`). El hilo t del proceso r es el trabajador global g = r·T + t y prueba g, g + pT, g + 2pT, …: el mismo intercalado, ahora con w = p·T trabajadores.




```
MPI_Init_thread(MPI_THREAD_FUNNELED)             # solo el hilo principal llama a MPI
mpi_search_setup; T = omp_get_max_threads() del proceso 0, MPI_Bcast(T)
antena: MPI_Irecv; MPI_Barrier; inicio = MPI_Wtime()
#pragma omp parallel num_threads(T) reduction(+: llaves)
    g = r·T + t;  workspace propio
    para k = g, g + pT, g + 2pT, ... < N:
        cada 4096 llaves:
            si t == 0 y MPI_Test(antena): stop = 1     # aviso de otro proceso
            si stop: salir
        si try_key(k): critical { guardar k, iteraciones }; stop = 1; salir
si un hilo la halló: MPI_Send(aviso) a todos        # fuera de la región paralela
MPI_Wait(antena); MPI_Reduce(llaves); reporte con hilos = T
```

Decisiones. `try_key` es reentrante (la agenda de subllaves de DES es local), así que basta un `workspace` por hilo. La bandera de parada es un `atomic_int` de C11; los hilos la leen cada 4096 llaves, igual que el sondeo MPI. Con `MPI_THREAD_FUNNELED` solo el hilo 0 (el principal) llama a `MPI_Test`, y el aviso sale después de la región paralela: no hace falta que el MPI instalado soporte llamadas desde varios hilos. T lo decide el proceso 0 y lo difunde, para que todos usen el mismo paso p·T.

Ojo al medir. Open MPI 5 ata cada proceso a **un** núcleo por defecto: los T hilos de un proceso se pelean ese núcleo y el híbrido no escala (en el i9, con 4 procesos, 1, 2 y 4 hilos tardaban casi lo mismo). Por eso el híbrido se lanza con `mpirun --bind-to none`, y así lo hacen `verificar.sh` y `campana.sh`. En macOS Open MPI no ata procesos, así que allá no cambia nada.

Quién la halla. El hilo g = k mod pT, tras ⌊k/(pT)⌋ + 1 iteraciones:

> S_iter = (k + 1) / (⌊k/(pT)⌋ + 1) ≈ p·T, y nunca mayor que p·T.

Con T = 1 es exactamente el intercalado. Para E, C y F se usa w = p·T.

- **Ventajas:** usa todos los núcleos con pocos procesos MPI (menos mensajes y menos memoria que `intercalado -np p·T`); el speedup sigue siendo consistente para cualquier llave.
- **Desventajas:** un nivel más de sincronización; depende de cómo el lanzador ata los procesos; con más trabajadores que núcleos de rendimiento, los hilos que caen en núcleos de eficiencia frenan al resto, porque el reparto es estático.
- **Fuga que domina:** la del **hardware** (turbo y núcleos P/E) al crecer w, más la **comunicación η_c** del sondeo, que ahora solo hace un hilo por proceso.

Demostración de consistencia. Sale de tres comparaciones de la campaña (capítulo de análisis): (1) con T = 1, el híbrido tarda lo mismo que el intercalado; (2) con T = 2 y 4, el tiempo baja de forma pareja en las tres llaves; (3) frente a `intercalado -np p·T`, con los mismos trabajadores pero todos MPI, el tiempo es parecido.

Tabla teórica (56 bits, `-np 4`, C = 2^16)

Llave	Naive	Intercalado	Bloque-cíclico	Maestro-trabajador
a · 123456	1	4.00	2.13	2.13
b · 2^54 − 1	1	4	4	3
c · 2^54	1.8·10^16 (aparente)	4	4	3
fácil · 2^55 + 1	1.8·10^16 (aparente)	4	4	3
media · 2^55 + 2^53	5 (aparente)	4	4	3
difícil · ⌈2^56/7 + 2^56/13⌉	1	4	4	3

Cómo se sacan a mano, con bloque N/4 = 2^54:

- **Naive, media:** k = N/2 + N/8 cae en el bloque del proceso 2, que empieza en N/2, y avanza N/8 + 1 llaves: S = (5N/8 + 1)/(N/8 + 1) ≈ **5**.
- **Naive, difícil:** k ≈ 0.22·N < N/4, cae en el bloque del proceso 0 y hace las mismas k + 1 iteraciones que el secuencial: **S = 1**.
- **Naive, c y fácil:** c = N/4 es la primera llave del proceso 1 (1 iteración) y fácil = N/2 + 1 la segunda del proceso 2 (2 iteraciones): S ≈ N/4 ≈ **1.8·10^16**.

Con 28 bits, los bits de las mediciones, la tabla es la misma salvo c y fácil del naive (≈ 6.7·10^7): **los presets escalan con N** y la llave conserva su posición relativa, por eso medir con menos bits es representativo.

Lo que gana una alternativa: consistencia, no promedio

Si la llave fuera uniforme al azar en [0, N), el T_p **esperado** del naive y del intercalado sería el mismo: en ambos, el proceso que la tiene recorre en promedio la mitad de N/p llaves, E[T_p] = N/(2p), mientras que E[T_s] = N/2, así que el speedup esperado es p en los dos. El S > p del naive en c, fácil y media se compensa con las llaves que le dan S = 1.

Lo que gana un reparto alternativo es que **el speedup ya no depende de dónde cae la llave**: ≈ p (o p − 1) para cualquier llave, en vez de algo entre 1 y 10^16. Y si las llaves no son uniformes, por ejemplo llaves pequeñas, el naive da S = 1 y el intercalado S = p.

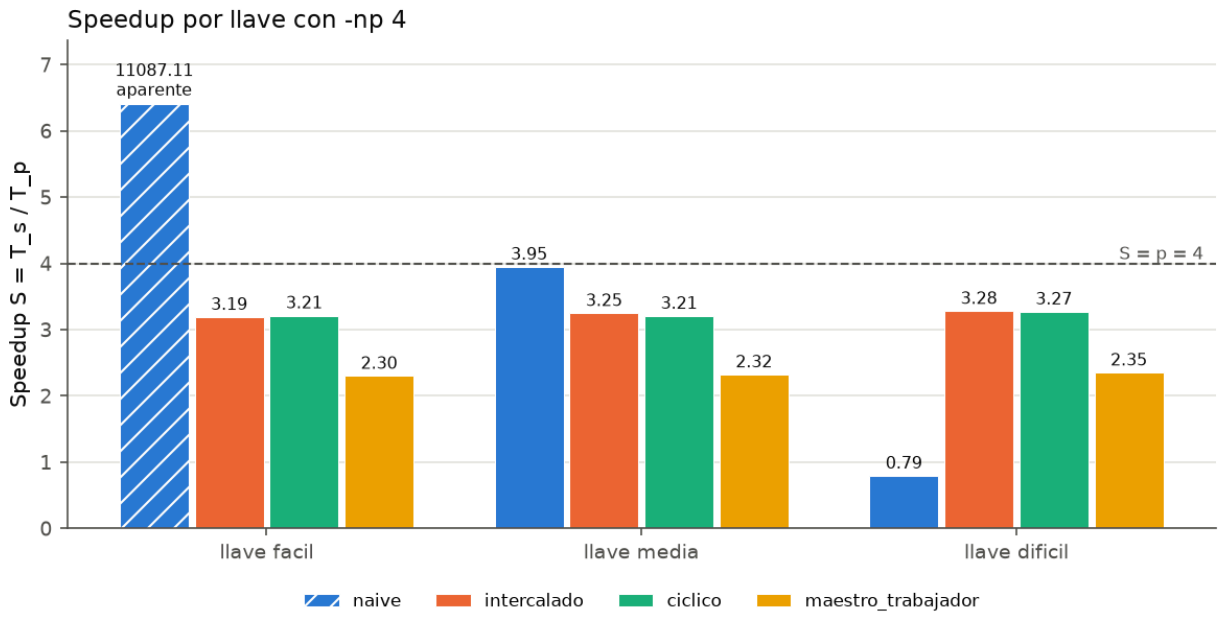
Análisis de speedup y del comportamiento según la llave

Los números de este capítulo salen de la campaña **preliminar** (i9-13980HX, 28 bits, 5 repeticiones, mediana; detalle en el Anexo 2 y en `resultados/analisis_escu.md`). Las fórmulas son las del curso: S = T_s / T_p, E = S / w, C = w · T_p, F = S / C, con w = p · h trabajadores (Barlas, 2015; Pacheco, 2011). T_s es siempre el programa `secuencial` con la misma llave y los mismos bits.

1. Lo teórico frente a lo medido

Llave (28 bits)		Naive	Intercalado	Bloque-cíclico	Maestro-trabajador
fácil · k = 134 217 729	S_iter (teoría)	6.7·10^7	4	4	3
	S medido	11 087 (aparente)	3.19	3.21	2.30
media · k = 167 772 160	S_iter (teoría)	5	4	4	3
	S medido	3.95	3.25	3.21	2.32
difícil · k = 58 996 804	S_iter (teoría)	1	4	4	3
	S medido	0.79	3.28	3.27	2.35

S_iter, que solo cuenta iteraciones, coincide **exactamente** con la teoría del capítulo de enfoques en todas las corridas: el reparto prueba las llaves que debe y quien halla la llave lo hace en el número de intentos previsto. La diferencia entre S_iter y el S medido es, entonces, pura pérdida de velocidad por llave (sección 4).



2. Por qué el naive es inconsistente

El tiempo del naive no depende de cuántas llaves hay, sino de **dónde cae la llave dentro del bloque del proceso que la tiene**. Con las mismas 4 llaves de proceso:

- **Fácil:** k = N/2 + 1 es la segunda llave del bloque del proceso 2. La halla en 2 intentos, 4 ms (casi todo es arranque y la primera ronda de sondeo), y el secuencial tarda 44.8 s: **S ≈ 11 000**. Con 56 bits sería ≈ 10^16.
- **Media:** k = N/2 + N/8 cae a la mitad del bloque del proceso 2, que recorre N/8 llaves: S_iter = 5. Medido, **S = 3.95**: los 4 procesos activos van cada uno al 79 % del ritmo del secuencial, y 5 × 0.79 ≈ 3.95.
- **Difícil:** k ≈ 0.22 N cae en el bloque del proceso 0, que hace las mismas k + 1 iteraciones que el secuencial: S_iter = 1 y medido **S = 0.79**. Es el peor caso: cuatro procesos, más lento que uno.

Entre llaves, el speedup del naive va de 0.79 a 11 000. Con las pruebas del enunciado (capítulo de pruebas B) pasa lo mismo: b y c son vecinas y una da S ≈ 1 y la otra S ≈ 10^12.

3. Por qué las alternativas son consistentes

El intercalado da S = 3.19–3.28 y el bloque-cíclico 3.21–3.27 en las tres llaves: una variación menor al 3 %, frente a cuatro órdenes de magnitud del naive. El maestro-trabajador da 2.30–2.35, también parejo, por debajo de los otros porque **el maestro no**

busca: con 3 trabajadores su techo es 3, y respecto a ese techo su eficiencia es 2.32 / 3 = 0.77, igual a la de los otros.

En todos ellos el que halla la llave hace $\lfloor k/p \rfloor + 1$ iteraciones (o el equivalente por chunks), así que $S \approx p$ sin importar dónde esté la llave.

4. El superlineal: aparente, nunca real

En esta campaña ningún programa alternativo superó $S = w$, y el análisis automático no levantó ninguna alerta. El único $S > p$ es el del naive con la llave fácil, y es **aparente**:

1. **No procesa llaves más rápido.** El speedup de rendimiento, S_rend = (llaves probadas / T_p) / ((k + 1) / T_s), compara llaves por segundo. Su máximo en toda la campaña es 3.37 con w = 4 (híbrido con 1 hilo) y 10.05 con w = 16: **siempre S_rend ≤ w**. En el naive con la llave media, S_rend = 3.16: el proceso 2 simplemente tuvo que recorrer 1/8 del espacio, no lo recorrió más rápido.
2. **Es el efecto de búsqueda** (ejercicio 9 de la clase): la llave queda al inicio de un bloque que no es el primero. De las tres causas de superlinealidad vistas en clase, la de caché no aplica (el cifrado mide 32 bytes y cabe en caché en ambos casos) y tampoco hay sobrecarga que se reduzca al partir.
3. **En promedio no existe.** Si la llave es uniforme en [0, N), el proceso que la tiene recorre en promedio la mitad de N/p llaves: $E[T_p] = N/(2p)$ contra $E[T_s] = N/2$, así que el speedup esperado del naive es p, igual que el del intercalado. Las llaves que le dan $S \gg p$ se compensan con las que le dan $S = 1$.

Si un enfoque alternativo diera $S > p$ habría que revisar, en este orden: T_s de otra llave o de otros bits, banderas de compilación distintas, el secuencial medido con la máquina ocupada, un reparto que se salta llaves (S_iter distinto a la teoría) o T_p medido después del aviso.

5. Fugas de eficiencia

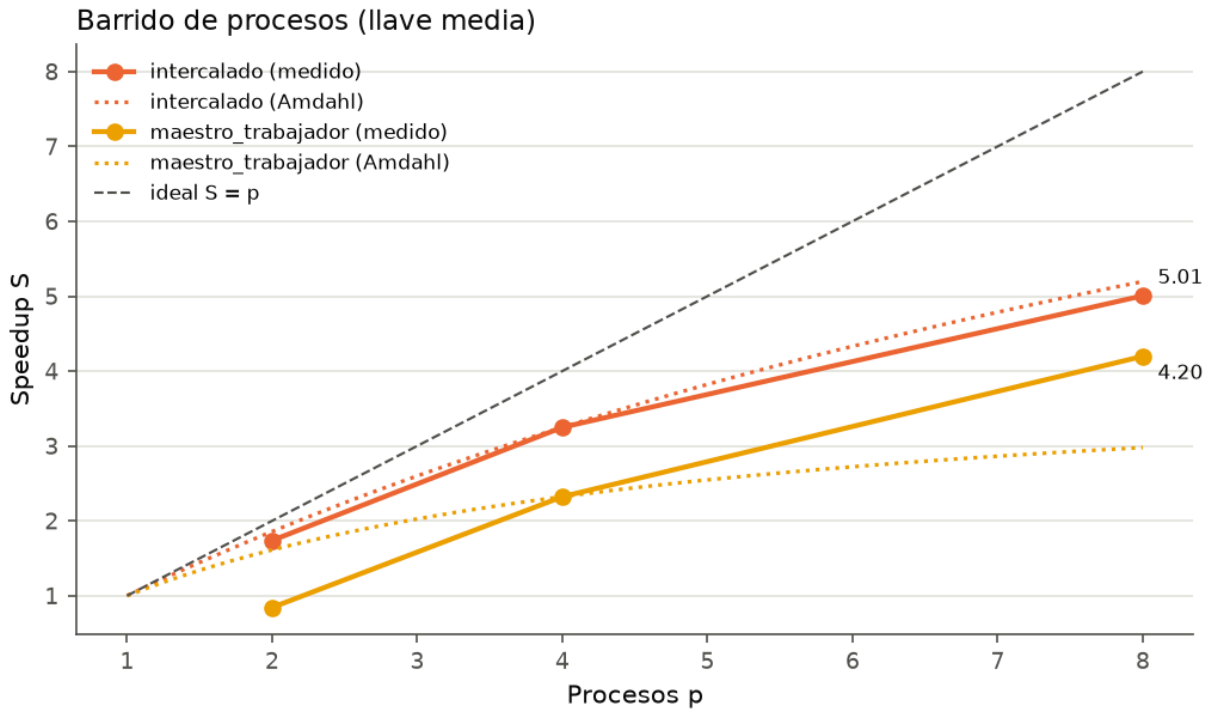
Con -np 4, el intercalado tiene $E \approx 0.81$ y además S_rend ≈ S (3.26 frente a 3.25): no se desperdician llaves, cada proceso prueba llaves más despacio que el secuencial solo. Eso ubica la pérdida:

Fuga	En nuestro problema
η_f parte secuencial	Leer, cifrar y difundir el texto: milisegundos. En el maestro-trabajador, el maestro entero (1/p del cómputo)
η_c comunicación	Un `MPI_Test` cada 4096 llaves y un aviso al final: despreciable. En el maestro-trabajador, un pedido por chunk, oculto en parte por el pedido adelantado

Fuga	En nuestro problema
η_b desbalance	Nulo en los repartos estáticos (todos reciben N/p llaves). En el naive, respecto a la llave: p − 1 procesos trabajan donde no está
η_r redundancia	Ninguna: cada llave se prueba una sola vez
Hardware	**La que domina.** Con 1 núcleo activo el i9 sube al turbo máximo; con 4 u 8 baja la frecuencia, y pasados 8 trabajadores se comparten núcleos (*hyperthreading*) o entran los de eficiencia

6. Barrido de procesos, Amdahl y efectividad

Programa	p	T_p (s)	S	E	C (s)	F
secuencial	1	56.57	1.00	1.00	56.57	0.018
intercalado	2	32.57	1.74	0.87	65.13	0.027
intercalado	4	17.41	3.25	0.81	69.65	0.047
intercalado	8	11.30	5.01	0.63	90.40	0.055
maestro-trabajador	2	67.13	0.84	0.42	134.27	0.006
maestro-trabajador	4	24.36	2.32	0.58	97.45	0.024
maestro-trabajador	8	13.48	4.20	0.52	107.82	0.039



Amdahl. Con el intercalado en p = 4: f_p = (1 − 1/3.25) / (1 − 1/4) = **0.923**, f_s = 0.077 y techo 1/f_s ≈ **13**. La predicción para p = 8 es 1/(0.077 + 0.923/8) = 5.20, y se

midió 5.01 (−3.7 %); para p = 2 predice 1.86 y se midió 1.74. Esa «f_s» no es código secuencial: es la pérdida de frecuencia al activar más núcleos, que crece con p en vez de quedarse fija. Por eso Amdahl, calibrado en p = 4, sobrestima los extremos.

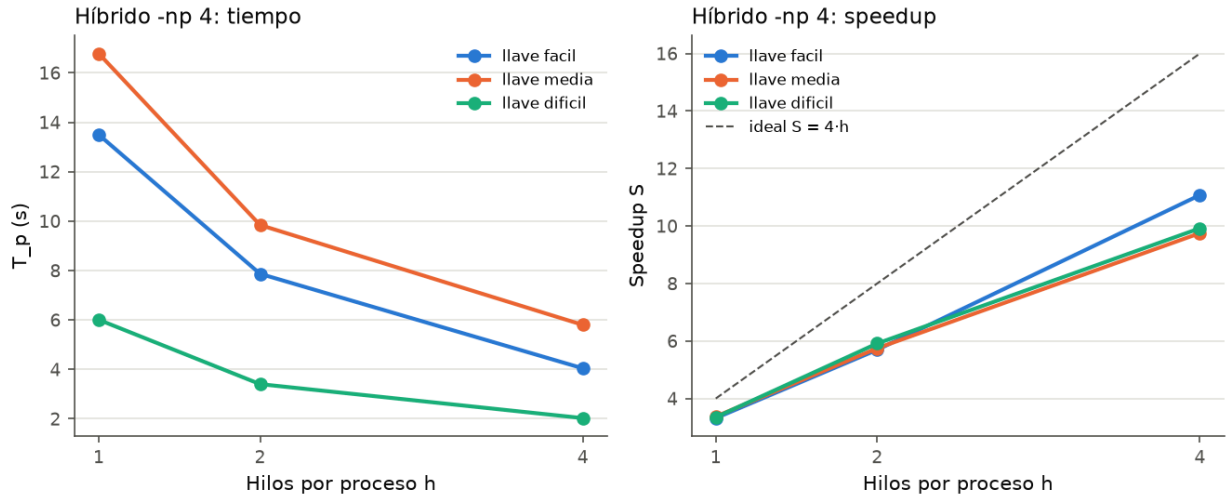
El maestro-trabajador no sigue a Amdahl porque su pérdida es estructural: con p procesos tiene p − 1 trabajadores. Con p = 2 hay uno solo y es más lento que el secuencial (S = 0.84); con p = 8 la fracción del maestro baja a 1/8 y S sube a 4.20, muy por encima de los 2.98 que predice Amdahl con la f_s de p = 4. Su ideal es S = p − 1, y E → 1 cuando p crece.

Gustafson. En fuerza bruta el problema crece solo: con S ≈ 3.25 se cubren 3.25 veces más llaves en el mismo tiempo, log2(3.25) ≈ **1.7 bits más**; con el híbrido de 16 trabajadores (S ≈ 10) son ≈ 3.3 bits más. Con f_s ≈ 0, el speedup escalado de Gustafson tiende a p.

Efectividad. F sigue creciendo hasta el p más grande medido (0.055 con 8 procesos en el intercalado y 0.105 con el híbrido de 16 trabajadores): todavía conviene agregar trabajadores, aunque la eficiencia baja de 0.87 a 0.63. Si se exige E ≥ 0.8, el punto es p = 4.

7. El híbrido: speedup consistente con y sin OpenMP

Llave	h = 1 (w = 4)	h = 2 (w = 8)	h = 4 (w = 16)
fácil	S = 3.32 · E = 0.83	S = 5.69 · E = 0.71	S = 11.08 · E = 0.69
media	S = 3.37 · E = 0.84	S = 5.75 · E = 0.72	S = 9.75 · E = 0.61
difícil	S = 3.35 · E = 0.84	S = 5.91 · E = 0.74	S = 9.92 · E = 0.62



1. **Con un hilo es el intercalado:** tarda 0.96–0.98 veces lo mismo (13.50 s frente a 14.04 s en fácil; 16.78 frente a 17.41 en media; 6.01 frente a 6.14 en difícil).

2. **Con más hilos baja parejo en las tres llaves:** S ≈ 5.7–5.9 con 2 hilos y ≈ 9.8–11.1 con 4. S_iter es exactamente 8 y 16, y S_rend nunca pasa de w.
3. **Frente a `intercalado -np 8`** (los mismos 8 trabajadores, todos MPI) el híbrido 4 × 2 tarda 9.84 s contra 11.30 s: un poco mejor, porque los hilos comparten el proceso y no hay ocho procesos sondeando MPI.

La eficiencia baja con 16 trabajadores porque el i9 tiene 8 núcleos de rendimiento: pasados 8 trabajadores, los demás comparten núcleo por *hyperthreading* o caen en núcleos de eficiencia, más lentos, y el reparto estático hace que todos esperen al más lento. Aun así, el speedup crece de forma consistente con w.

8. De 28 a 56 bits

Al ritmo medido del secuencial, 167 772 161 llaves en 56.57 s ≈ **2.97·10^6 llaves/s**:

- T_s(2^56) = 2^56 / 2.97·10^6 ≈ 2.43·10^10 s ≈ **770 años**.
- El intercalado con 4 procesos (S = 3.25) bajaría eso a ≈ 237 años, y el híbrido de 16 trabajadores (S ≈ 9.75) a ≈ 79 años.
- Para recorrer 2^56 llaves en un día harían falta ≈ 2.43·10^10 / 86 400 ≈ **281 000 núcleos** con eficiencia 1.

Como los presets escalan con N, S_iter con 28 bits es el mismo que con 56 (salvo c y fácil del naive, que valen ≈ N/4), y por eso estas mediciones representan el problema completo.

Conclusiones y recomendaciones

Conclusiones

1. **El reparto decide el speedup, no el número de procesos.** Con el mismo código de búsqueda y los mismos 4 procesos, el naive va de S = 0.79 (llave difícil) a S = 11 000 aparente (llave fácil con 28 bits; ≈ 10^16 con 56), mientras que el intercalado y el bloque-cíclico dan S = 3.19–3.28 con **cualquier** llave. El tiempo del naive depende de dónde cae la llave dentro del bloque del proceso que la tiene; el de los repartos alternativos, solo de k/p.
2. **El proyecto no es superlineal.** Los S > p del naive son un **efecto de búsqueda**: el proceso que tiene la llave empieza cerca de ella, pero no prueba llaves más rápido. El speedup de rendimiento (llaves por segundo) nunca pasa de p en ningún programa, y si se promedia sobre todas las llaves el speedup esperado del naive es p, igual que el del intercalado. El análisis automático lo comprueba en cada campaña: ningún enfoque alternativo da S > w.
3. **Lo que gana un enfoque alternativo es consistencia.** El intercalado y el bloque-cíclico sostienen E ≈ 0.80–0.82 en las tres llaves; el maestro-trabajador queda en S ≈ 2.3 con 4 procesos (0.77 de su techo de p − 1 = 3) porque el maestro no busca, a cambio de balancear solo la carga cuando los núcleos son desiguales.
4. **La «parte secuencial» de Amdahl no es código secuencial.** Leer y cifrar el texto tarda milisegundos; la f_s que se despeja del speedup medido es pérdida efectiva: el turbo que baja con más núcleos activos, los núcleos de eficiencia, el sondeo de MPI y, en el maestro-trabajador, el propio maestro. Por eso Amdahl, calibrado con un p, no predice bien los demás.
5. **La fuerza bruta escala al estilo de Gustafson.** El trabajo crece solo (cada bit duplica el espacio), y con w trabajadores en el mismo tiempo se cubren log2(w) bits más. Recorrer las 2^56 llaves en un día exigiría unos 281 000 núcleos al ritmo medido de un solo núcleo (2.97·10^6 llaves/s); en uno solo serían unos 770 años.
6. **El híbrido MPI + OpenMP da un speedup consistente con la versión sin OpenMP.** Con un hilo es el intercalado (0.96–0.98 de su tiempo); con 2 y 4 hilos el speedup sube a ≈ 5.8 y ≈ 10 de forma pareja en las tres llaves, y con 8 trabajadores queda incluso algo mejor que el intercalado puro, siempre que los procesos no queden atados a un núcleo.

Retos y cómo se resolvieron con Open MPI

Reto	Solución
`rpc/des_crypt.h` ya no existe en glibc moderna	DES de OpenSSL (`DES_ecb_encrypt`), verificado contra `openssl enc -des-ecb`
Errores impresos por todos los procesos y procesos colgados	Solo el proceso 0 lee y valida; difunde un encabezado de estado con `MPI_Bcast` y todos salen limpios
El aviso de llave no llegaba mientras se calculaba	`MPI_Irecv` antes de buscar y `MPI_Test` cada 4096 llaves, que hace avanzar la recepción
La prueba b con 56 bits no termina (≈ 150 años)	Límite `-t` con un centinela que el proceso 0 difunde al vencer, y la misma prueba con 28 bits
Medir con 56 bits es imposible	Presets que escalan con N = 2^bits: la llave conserva su posición relativa y S_iter
El maestro-trabajador se colgaba si los chunks se acababan antes de hallar la llave	Protocolo con dos invariantes: una respuesta por pedido y un único mensaje final por trabajador
El híbrido no escalaba con más hilos	Open MPI 5 ata cada proceso a un núcleo: se lanza con `--bind-to none`
Varios hilos y un MPI sin soporte multihilo completo	`MPI_THREAD_FUNNELED`: solo el hilo principal llama a MPI

Aprendizajes

- Un speedup aislado no dice nada sin el caso que se midió: hay que reportar la llave, los bits, el T_s del secuencial de verdad y la mediana de varias corridas.
- El speedup de rendimiento y el speedup por iteraciones separan lo que aporta el paralelismo de lo que aporta la suerte de dónde cae la llave.
- Los detalles del lanzador (`mpirun`) y del hardware (núcleos de rendimiento y de eficiencia, turbo) pesan tanto como el algoritmo cuando la comunicación es mínima.
- Un protocolo de mensajes necesita invariantes explícitas; los casos borde (pocos chunks, hallazgo al final) son justamente los que no aparecen con los parámetros por defecto.

Recomendaciones

- Para fuerza bruta con llaves de distribución desconocida, usar un reparto **intercalado** o **bloque-cíclico**: dan el speedup esperado para cualquier llave sin comunicación extra.
- Si los núcleos son desiguales o la máquina es compartida, usar **maestro-trabajador** con suficientes procesos para que el maestro pese poco ($(p-1)/p \to 1$).

- Medir siempre con el secuencial real, las mismas banderas de compilación, la máquina sin carga y al menos 5 repeticiones; tratar cualquier $S > p$ como un error hasta explicarlo.
- En híbridos, revisar el *binding* (`mpirun --report-bindings`) y no pedir más hilos que núcleos de rendimiento.
- Para 56 bits reales, el paso siguiente es un clúster o aceleradores (GPU): el problema es vergonzosamente paralelo y escala con el número de núcleos.

Referencias

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the AFIPS Spring Joint Computer Conference* (pp. 483–485). ACM. https://doi.org/10.1145/1465482.1465560

Barlas, G. (2015). *Multicore and GPU programming: An integrated approach*. Morgan Kaufmann.

Electronic Frontier Foundation. (1998). *Cracking DES: Secrets of encryption research, wiretap politics & chip design*. O'Reilly.

Fedora Project. (s. f.). *Changes/SunRPCRemoval: Removal of Sun RPC interfaces from glibc*. Fedora Project Wiki. https://fedoraproject.org/wiki/Changes/SunRPCRemoval

Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM, 31*(5), 532–533. https://doi.org/10.1145/42411.42415

ISO/IEC. (2011). *ISO/IEC 9899:2011. Information technology — Programming languages — C*.

Message Passing Interface Forum. (2023). *MPI: A message-passing interface standard (Version 4.1)*. https://www.mpi-forum.org/docs/

National Institute of Standards and Technology. (1999). *FIPS PUB 46-3: Data Encryption Standard (DES)*. U.S. Department of Commerce.

National Institute of Standards and Technology. (2005). Announcing approval of the withdrawal of Federal Information Processing Standard (FIPS) 46-3, Data Encryption Standard (DES). *Federal Register, 70*(96), 28907–28908.

Open MPI Project. (s. f.). *mpirun / prterun: Mapping, ranking, and binding* (Open MPI 5.0.x documentation). https://docs.open-mpi.org/

OpenMP Architecture Review Board. (2021). *OpenMP application programming interface (Version 5.2)*. https://www.openmp.org/specifications/

OpenSSL Project. (s. f.). *DES_random_key, DES_set_key, DES_ecb_encrypt: DES encryption* (OpenSSL 3 manual pages). https://docs.openssl.org/3.0/man3/DES_random_key/

Pacheco, P. (2011). *An introduction to parallel programming*. Morgan Kaufmann.

Stallings, W. (2020). *Cryptography and network security: Principles and practice* (8.ª ed.). Pearson.

Weimer, F. (2020, julio). *[PATCH] Remove --enable-obsolete-rpc configure flag* [Mensaje en lista de correo]. libc-alpha. https://inbox.sourceware.org/libc-alpha/20200711181508.GA9224@x230/t

Anexo 1 · Catálogo de funciones y librerías

Todas las funciones del proyecto, incluidas las `static`, agrupadas por archivo. Cada una lleva sus entradas y salidas (nombre, tipo y uso) y una descripción. Al final están las funciones de librería que usamos: OpenSSL, Open MPI, la libc, OpenMP y los módulos de Python del análisis.

`des_utils.c` · DES sobre OpenSSL

`build_des_key`

Entradas	Tipo	Uso
`key56`	`uint64_t`	Llave numérica; solo se usan sus 56 bits bajos
`out`	`unsigned char[8]`	Buffer donde se escribe la llave DES

Salidas	Tipo	Uso
`out`	`unsigned char[8]`	8 bytes con 7 bits de llave y 1 de paridad cada uno

Descripción. Reparte los 56 bits en grupos de 7, uno por byte (bits 1..7), y completa la paridad impar con `DES_set_odd_parity`, igual que el `decrypt()` del `bruteforce.c` original.

`run_des` (`static`)

Entradas	Tipo	Uso
`key`	`uint64_t`	Llave de 56 bits
`buffer`	`unsigned char *`	Datos que se cifran o descifran en el mismo lugar
`length`	`size_t`	Bytes de `buffer`, múltiplo de 8
`direction`	`int`	`DES_ENCRYPT` o `DES_DECRYPT`

Salidas	Tipo	Uso
`buffer`	`unsigned char *`	Datos ya transformados

Descripción. Arma la llave, genera el *key schedule* con `DES_set_key_unchecked` (sin rechazar llaves débiles) y aplica `DES_ecb_encrypt` bloque a bloque.

`encrypt_des` y `decrypt_des`

Entradas	Tipo	Uso
`key`	`uint64_t`	Llave de 56 bits

Entradas	Tipo	Uso
`buffer`	`unsigned char *`	Texto plano (cifrar) o cifrado (descifrar)
`length`	`size_t`	Bytes de `buffer`, múltiplo de 8

Salidas	Tipo	Uso
`buffer`	`unsigned char *`	Resultado, en el mismo buffer

Descripción. Cifran o descifran con DES en modo ECB llamando a `run_des`. Reemplazan a `encrypt` y `decrypt` del `bruteforce.c`.

`try_key`

Entradas	Tipo	Uso
`key`	`uint64_t`	Llave candidata
`cipher`	`const unsigned char *`	Texto cifrado original (no se modifica)
`length`	`size_t`	Bytes del cifrado
`phrase`	`const char *`	Frase clave conocida
`workspace`	`unsigned char *`	Buffer de `length + 1` bytes reservado una sola vez

Salidas	Tipo	Uso
retorno	`int`	1 si el descifrado contiene la frase, 0 si no

Descripción. Copia el cifrado al `workspace` con `memcpy`, lo descifra con la llave candidata y busca la frase con `strstr`.

`padded_length`

Entradas	Tipo	Uso
`length`	`size_t`	Longitud original

Salidas	Tipo	Uso
retorno	`size_t`	Longitud redondeada al siguiente múltiplo de 8

Descripción. Calcula cuántos bytes ocupa el texto con relleno de ceros hasta completar bloques DES.

`options.c` · Parámetros

`print_usage` (`static`)

Entradas	Tipo	Uso
`program`	`const char *`	Nombre del programa
`flags`	`int`	Opciones extra que admite (`OPT_OUTPUT`, `OPT_CHUNK`)

Salidas	Tipo	Uso
(stderr)	texto	Ayuda del programa

Descripción. Imprime la forma de uso. `-c` y `-o` solo aparecen en los programas que las admiten.

`parse_u64` (`static`)

Entradas	Tipo	Uso
`text`	`const char *`	Número en decimal o en hexadecimal con `0x`; acepta una `L` final
`value`	`uint64_t *`	Destino del número

Salidas	Tipo	Uso
retorno	`int`	0 si la conversión fue completa, -1 si hay basura, signo o desbordamiento
`*value`	`uint64_t`	El número convertido

Descripción. Envuelve `strtoull` y rechaza todo lo que no sea un número completo.

`resolve_key`

Entradas	Tipo	Uso
`text`	`const char *`	Llave escrita en `-k`: número o preset
`key_bits`	`int`	Bits del espacio de búsqueda
`key_out`	`uint64_t *`	Destino de la llave

Salidas	Tipo	Uso
retorno	`int`	0 si es válida y cabe en 2^bits, -1 si no
`*key_out`	`uint64_t`	La llave resuelta

Descripción. Traduce los presets `a`, `b`, `c`, `facil`, `media` y `dificil` (escalados con N = 2^bits) o convierte el número, y valida que la llave sea menor que N.

`parse_options`

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char **`	Argumentos del programa
`program`	`const char *`	Nombre para la ayuda
`flags`	`int`	Opciones extra admitidas (`OPT_OUTPUT`, `OPT_CHUNK`)
`opts`	`program_options_t *`	Destino de los parámetros

Salidas	Tipo	Uso
retorno	`int`	`OPTIONS_OK`, `OPTIONS_HELP` u `OPTIONS_ERROR`
`*opts`	`program_options_t`	Archivo, llave, frase, bits, límite de tiempo, chunk y salida

Descripción. Lee las opciones con `getopt` y las valida: rangos de `-b`, `-t` y `-c`, que `-c` sea menor que `-b`, que la frase no esté vacía y que la llave sea válida. Los mensajes de error salen en español.

`problem.c` · Problema y reporte

`read_text_file` (`static`)

Entradas	Tipo	Uso
`path`	`const char *`	Ruta del `.txt`
`length_out`	`size_t *`	Destino de la longitud

Salidas	Tipo	Uso
retorno	`char *`	Texto terminado en `'\0'` (liberar con `free`), o `NULL` si falla
`*length_out`	`size_t`	Bytes útiles, sin los saltos de línea finales

Descripción. Lee el archivo completo, hasta 64 KiB. Rechaza archivos vacíos, demasiado grandes o con bytes nulos.

`problem_prepare`

Entradas	Tipo	Uso
`opts`	`const program_options_t *`	Parámetros validados
`problem`	`crack_problem_t *`	Destino del problema

Salidas	Tipo	Uso
retorno	`int`	0 si todo salió bien, -1 si no

Salidas	Tipo	Uso
`*problem`	`crack_problem_t`	Texto, cifrado con relleno, frase y tamaño del espacio

Descripción. Carga el texto, verifica que contenga la frase clave y lo cifra con la llave privada.

`problem_release`

Entradas	Tipo	Uso
`problem`	`crack_problem_t *`	Problema, completo o a medio construir

Salidas	Tipo	Uso
`*problem`	`crack_problem_t`	Punteros liberados y en `NULL`

Descripción. Libera el texto, el cifrado y la frase. Se puede llamar más de una vez.

`problem_print_header`

Entradas	Tipo	Uso
`program`	`const char *`	Nombre del programa
`opts`	`const program_options_t *`	Parámetros
`problem`	`const crack_problem_t *`	Problema preparado
`processes`	`int`	Número de procesos

Salidas	Tipo	Uso
(stdout)	texto	Encabezado de la corrida

Descripción. Imprime el archivo, la frase, la llave y el tamaño del espacio antes de buscar.

`problem_print_result`

Entradas	Tipo	Uso
`program`	`const char *`	Nombre del programa
`opts`	`const program_options_t *`	Llave real y límite de tiempo
`problem`	`const crack_problem_t *`	Para descifrar y mostrar el texto
`result`	`const search_result_t *`	Resultado de la búsqueda

Salidas	Tipo	Uso
(stdout)	texto	Reporte y línea `CSV,` de 11 columnas

Descripción. Muestra la llave hallada, quién la halló y en cuántas iteraciones, el texto descifrado, las llaves probadas, el ritmo y el tiempo, y deja la línea CSV para la bitácora.

`print_hex`

Entradas	Tipo	Uso
`label`	`const char *`	Etiqueta de la línea
`bytes`	`const unsigned char *`	Datos
`length`	`size_t`	Bytes a imprimir

Salidas	Tipo	Uso
(stdout)	texto	Bytes en hexadecimal, en grupos de 8

Descripción. Imprime un buffer en hexadecimal, separado por bloques DES.

`wall_seconds`

Entradas	Tipo	Uso
—	—	—

Salidas	Tipo	Uso
retorno	`double`	Segundos del reloj monotónico

Descripción. Reloj de pared del secuencial (`clock_gettime(CLOCK_MONOTONIC)`).

`cifrar.c` · Parte B.1

`write_cipher_file` (`static`)

Entradas	Tipo	Uso
`path`	`const char *`	Archivo de salida
`bytes`	`const unsigned char *`	Texto cifrado
`length`	`size_t`	Bytes a escribir

Salidas	Tipo	Uso
retorno	`int`	0 si se escribió completo, -1 si no

Descripción. Guarda el cifrado en binario y revisa tanto `fwrite` como `fclose`.

`main` (cifrar)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char **`	`-f`, `-k`, `-p`, `-b`, `-o`

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS` si el descifrado coincide con el original
(stdout)	texto	Texto, cifrado en hexadecimal, descifrado y contraejemplo

Descripción. Cifra el `.txt` con la llave dada, lo descifra de vuelta, lo compara con el original y muestra qué sale con la llave vecina.

`secuencial.c` · Parte A.3

`main` (secuencial)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char **`	`-f`, `-k`, `-p`, `-b`, `-t`

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS` si la halló o si se usó `-t`
(stdout)	texto	Reporte y línea CSV

Descripción. Recorre las llaves 0, 1, 2, … hasta que `try_key` acierta, o hasta que vence el límite de tiempo, que revisa cada 4096 llaves.

`search_mpi.c` · Módulo común MPI

`validate_processes` (`static`)

Entradas	Tipo	Uso
`size`	`int`	Número de procesos
`min_processes`	`int`	Mínimo que necesita el programa
`problem`	`const crack_problem_t *`	Para comparar con 2^bits

Salidas	Tipo	Uso
retorno	`int`	0 si el número de procesos sirve, -1 si no

Descripción. El proceso 0 rechaza menos procesos que el mínimo (el maestro-trabajador pide 2) o más procesos que llaves.

`share_problem` (`static`)

Entradas	Tipo	Uso
`search`	`mpi_search_t *`	Búsqueda; en el proceso 0 ya preparada
`status`	`int`	Estado del proceso 0: listo, ayuda o error

Salidas	Tipo	Uso
retorno	`int`	El estado del proceso 0, igual en todos
`*search`	`mpi_search_t`	En los demás procesos: cifrado, frase, bits, límite y chunk

Descripción. Difunde con `MPI_Bcast` un encabezado de 6 enteros (estado, bits, longitudes, límite de tiempo y chunk) y luego el cifrado y la frase. Si el estado no es «listo», todos salen sin quedar esperando.

`mpi_search_setup`

Entradas	Tipo	Uso
`argc`, `argv`	`int *`, `char ***`	Argumentos (se pasan a `MPI_Init`)
`program`	`const char *`	Nombre del programa
`option_flags`	`int`	0 u `OPT_CHUNK`
`min_processes`	`int`	Mínimo de procesos
`search`	`mpi_search_t *`	Destino del estado

Salidas	Tipo	Uso
retorno	`int`	`SEARCH_READY`, `SEARCH_HELP` o `SEARCH_ERROR`
`*search`	`mpi_search_t`	Rango, procesos, parámetros, problema y `workspace`

Descripción. Inicia MPI con `MPI_Init`, salvo que ya esté iniciado (`MPI_Initialized`; el híbrido lo inicia antes con `MPI_Init_thread`). El proceso 0 lee los parámetros y el archivo y valida el número de procesos; luego se reparte el problema, el proceso 0 imprime el encabezado y cada proceso reserva su `workspace`. Si no hay que buscar, cierra MPI antes de volver.

`mpi_search_release`

Entradas	Tipo	Uso
`search`	`mpi_search_t *`	Búsqueda

Salidas	Tipo	Uso
—	—	Memoria liberada y MPI cerrado

Descripción. Libera el `workspace` y el problema y llama a `MPI_Finalize`.

`mpi_search_begin`

Entradas	Tipo	Uso
—	—	—

Salidas	Tipo	Uso
retorno	`double`	`MPI_Wtime` justo después de la barrera

Descripción. Sincroniza a todos con `MPI_Barrier` y marca el inicio de la búsqueda.

`notify_all` (`static`)

Entradas	Tipo	Uso
`message`	`const uint64_t[3]`	{llave, proceso, iteraciones}
`size`	`int`	Número de procesos

Salidas	Tipo	Uso
—	—	Un `MPI_Send` a cada proceso

Descripción. Envía el aviso a todos, incluido uno mismo, para que todos los `MPI_Irecv` se completen.

`announce_timeout` (`static`)

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena del proceso 0
`size`	`int`	Número de procesos

Salidas	Tipo	Uso
`listener->timeout_sent`	`int`	Queda en 1

Descripción. Avisa a todos con la llave centinela `UINT64_MAX` que venció el límite de tiempo.

`found_listener_start`

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Destino de la antena
`search`	`const mpi_search_t *`	Para el límite de tiempo

Salidas	Tipo	Uso
`*listener`	`found_listener_t`	`MPI_Irecv` puesto, reloj iniciado y plazo (solo en el proceso 0)

Descripción. Pone la antena antes de buscar, sincroniza y arranca el reloj.

`found_listener_poll`

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena
`search`	`const mpi_search_t *`	Número de procesos

Salidas	Tipo	Uso
retorno	`int`	1 si ya llegó un aviso, 0 si no

Descripción. Revisa la antena con `MPI_Test`; el proceso 0 además manda el aviso de límite si el plazo ya venció.

`found_listener_announce`

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Rango y número de procesos
`key`	`uint64_t`	Llave hallada
`iterations`	`uint64_t`	Llaves que probó este proceso

Salidas	Tipo	Uso
—	—	Aviso enviado a todos

Descripción. Arma {llave, rango, iteraciones} y lo envía con `notify_all`.

`found_listener_try_key`

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena
`search`	`const mpi_search_t *`	Cifrado, frase y `workspace`
`key`	`uint64_t`	Llave candidata
`iterations`	`uint64_t *`	Contador de llaves de este proceso

Salidas	Tipo	Uso
retorno	`int`	1 si hay que parar (la halló este proceso u otro avisó)
`*iterations`	`uint64_t`	Incrementado si se probó la llave

Descripción. Es un paso del bucle de los repartos estáticos: sondea cada 4096 llaves, prueba la llave y, si acierta, avisa a todos.

`found_listener_finish`

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena
`search`	`const mpi_search_t *`	Número de procesos
`result`	`search_result_t *`	Destino del resultado

Salidas	Tipo	Uso
`*result`	`search_result_t`	Hallazgo, llave, proceso, iteraciones y tiempo

Descripción. El proceso 0 sigue vigilando el plazo, con pausas de 1 ms, mientras espera. Luego todos cierran la antena con `MPI_Wait`, toman el tiempo y arman el resultado.

`mpi_search_report`

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Búsqueda
`program`	`const char *`	Nombre del programa
`result`	`const search_result_t *`	Resultado (se usa en el proceso 0)
`own_iterations`	`uint64_t`	Llaves que probó este proceso

Salidas	Tipo	Uso
(stdout)	texto	Reporte y CSV, impresos por el proceso 0

Descripción. Suma las llaves de todos con `MPI_Reduce` y el proceso 0 llama a `problem_print_result`.

Programas MPI

`main` (naive, intercalado, ciclico)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char **`	`-f`, `-k`, `-p`, `-b`, `-t` (y `-c` en ciclico)

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS`, o `EXIT_FAILURE` si los parámetros son inválidos

Salidas	Tipo	Uso
(stdout)	texto	Reporte y CSV del proceso 0

Descripción. Los tres siguen el mismo esqueleto: `mpi_search_setup` → `found_listener_start` → bucle con `found_listener_try_key` → `found_listener_finish` → `mpi_search_report` → `mpi_search_release`. Solo cambia el recorrido: - naive: bloque contiguo `[r·N/p, (r+1)·N/p)`; - intercalado: `r, r+p, r+2p, …`; - ciclico: chunks `r, r+p, …` de 2^c llaves, cada uno en orden.

`run_master` (`static`, maestro_trabajador)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Espacio, chunk, límite y procesos
`result`	`search_result_t *`	Destino del resultado

Salidas	Tipo	Uso
`*result`	`search_result_t`	Hallazgo, proceso, iteraciones y T_p

Descripción. Atiende pedidos con `MPI_Recv(ANY_SOURCE)`. A cada pedido (ASK) responde exactamente un mensaje: un chunk (`TAG_WORK`) o `TAG_STOP`. Responde STOP desde que llega el hallazgo (y ahí toma T_p), cuando vence el plazo o cuando se acaban los chunks. Los mensajes finales (FOUND o DONE) no llevan respuesta y descuentan un trabajador activo; termina cuando recibió el mensaje final de todos.

`search_chunk` (`static`, maestro_trabajador)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Cifrado, frase y `workspace`
`work`	`const uint64_t[2]`	{primera llave, una más allá de la última}
`iterations`	`uint64_t *`	Contador de llaves del trabajador
`found_key`	`uint64_t *`	Destino de la llave

Salidas	Tipo	Uso
retorno	`int`	1 si la halló, 0 si no

Descripción. Prueba en orden todas las llaves de un chunk.

`run_worker` (`static`, maestro_trabajador)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Búsqueda

Salidas	Tipo	Uso
retorno	`uint64_t`	Llaves que probó el trabajador

Descripción. Pide un chunk y, apenas lo recibe, pide el siguiente por adelantado (`MPI_Send` + `MPI_Irecv`). Prueba el actual y espera el próximo con `MPI_Wait`. Si halla la llave, envía de inmediato el mensaje final FOUND y recibe (y descarta) la respuesta pendiente; si recibe STOP sin hallarla, envía el mensaje final DONE.

`main` (maestro_trabajador)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char **`	`-f`, `-k`, `-p`, `-b`, `-t`, `-c`

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS`, o `EXIT_FAILURE` con parámetros inválidos o `-np 1`
(stdout)	texto	Reporte y CSV del proceso 0

Descripción. Prepara la búsqueda con mínimo 2 procesos. El proceso 0 corre `run_master` y los demás `run_worker`; al final, `mpi_search_report` con 0 llaves aportadas por el maestro.

`hibrido.c` · MPI + OpenMP

`init_threaded_mpi` (`static`)

Entradas	Tipo	Uso
`argc`, `argv`	`int *`, `char ***`	Argumentos (se pasan a `MPI_Init_thread`)

Salidas	Tipo	Uso
retorno	`int`	0 si MPI da al menos `MPI_THREAD_FUNNELED`, −1 si no (ya avisó y cerró MPI)

Descripción. Inicia MPI con `MPI_Init_thread` pidiendo `MPI_THREAD_FUNNELED` (solo el hilo principal llama a MPI) y valida el nivel que el MPI instalado concede en `provided`.

`share_threads` (`static`)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Rango, procesos y espacio de llaves

Salidas	Tipo	Uso
retorno	`int`	Hilos por proceso T, o −1 si no sirven

Descripción. El proceso 0 lee T con `omp_get_max_threads` (es decir, de `OMP_NUM_THREADS`), valida 1 ≤ T ≤ 256 y que p·T no supere las llaves del espacio, y lo difunde con `MPI_Bcast` para que todos usen el mismo T aunque el entorno de algún proceso difiera.

`search_thread` (`static`)

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena del proceso; solo la toca el hilo 0
`search`	`const mpi_search_t *`	Problema, rango y procesos
`threads`	`int`	Hilos por proceso T
`shared`	`thread_shared_t *`	Bandera de parada y hallazgo del proceso

Salidas	Tipo	Uso
retorno	`uint64_t`	Llaves que probó este hilo
`*shared`	`thread_shared_t`	Llave e iteraciones si este hilo la halló; `stop` levantada

Descripción. Cuerpo de la región paralela. El hilo t del proceso r es el trabajador global g = r·T + t y prueba g, g + pT, g + 2pT, … con su propio `workspace`. Cada `POLL_INTERVAL` llaves revisa la bandera atómica `stop`; el hilo 0 además llama a `found_listener_poll` (`MPI_Test`) y levanta la bandera si llegó un aviso de otro proceso. El hilo que halla la llave la guarda en una sección `critical` y levanta la bandera.

`main` (hibrido)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char **`	`-f`, `-k`, `-p`, `-b`, `-t`
`OMP_NUM_THREADS`	variable de entorno	Hilos por proceso (la lee el proceso 0)

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS`, o `EXIT_FAILURE` con parámetros o hilos inválidos
(stdout)	texto	Reporte y CSV del proceso 0, con `hilos = T`

Descripción. Inicia MPI con hilos, prepara la búsqueda con `mpi_search_setup`, acuerda T y pone la antena. Abre la región paralela (`#pragma omp parallel` con `reduction(+)` de las llaves probadas) y, ya fuera de ella, el hilo principal hace el `found_listener_announce` si un hilo halló la llave (FUNNELED). Cierra con `found_listener_finish` y `mpi_search_report`.

`scripts/analizar.py` · Análisis de la campaña

Funciones de Python que convierten `resultados/campana_<host>_*.csv` en `resultados/analisis_<host>.md` y las gráficas de `docs/img/`.

Función	Entradas	Salidas	Descripción
`Group` (clase)	caso, programa, procesos, hilos, bits, llave real	objeto con listas de tiempos, iteraciones y llaves probadas	Repeticiones de un mismo caso; `time`, `finder_iterations` y `keys_tried` dan la **mediana** y `workers` = p·h
`case_rank`, `program_rank`	`str` caso o programa	`int`	Orden fijo de llaves y programas en tablas y gráficas
`host_from_paths`	`list[str]` rutas	`str` host	Saca el host del nombre `campana_<host>_<fecha>.csv`
`load_groups`	`list[str]` rutas	grupos válidos, todos los grupos, `int` filas	Lee los CSV, agrupa y aparta las corridas con `llave_hallada != llave_real`
`sequential_times`	grupos	`dict` (caso, bits) → T_s	Mediana del `secuencial` por llave y bits
`metrics`	`Group`, `float` T_s	`dict` S, E, C, F, S_iter, S_rend	Las fórmulas de clase con w = p·h
`amdahl_fp`	`float` S, `int` p	`float` f_p	f_p = (1 − 1/S)/(1 − 1/p)
`amdahl_speedup`	`float` f_p, `float` p	`float` S	1/(f_s + f_p/p)
`gustafson_speedup`	`float` f_s, `int` p	`float` S	p − f_s(p − 1)
`ideal_speedup`	`Group`	`int`	p·h, o p − 1 en el maestro-trabajador
`note`	`Group`, métricas, lista de alertas	`str` nota	Regla del superlineal: «aparente (búsqueda)» en el naive; `REVISAR` si otro programa da S > w o cualquiera da S_rend > w (tolerancia 2 %)
`fmt`	`float`, dígitos	`str`	Número legible, en notación científica si es enorme o diminuto
`time_cell`, `label`	`Group`	`str`	Celda «mediana (mín–máx)» y nombre del programa con sus hilos
`main_table`	líneas, grupos, T_s, alertas	líneas de Markdown	Tabla de T_p, S, E, C, F, S_iter, S_rend y nota

Función	Entradas	Salidas	Descripción
`amdahl_section`	líneas, grupos, T_s, p principal, llave del barrido	`dict` programa → f_p	f_p, f_s, techo, Gustafson y bits extra; predicción de Amdahl frente al barrido
`style_axes`	eje de matplotlib	—	Rejilla y ejes discretos
`plot_speedup_by_key`	matplotlib, grupos, T_s, p, ruta	PNG	Barras de S por programa y llave con la línea S = p; el naive aparente se recorta y se rotula
`plot_sweep`	matplotlib, grupos, T_s, f_p, llave, ruta	PNG	S contra p con la ideal y la curva de Amdahl
`plot_hybrid`	matplotlib, grupos, T_s, ruta	PNG	Tiempo y speedup del híbrido contra los hilos
`plot_all`	grupos, T_s, f_p, p, llave, host, carpeta	`list[str]` rutas	Genera las gráficas si hay matplotlib; si no, avisa y sigue
`analyze`	rutas, salida, carpeta de imágenes	`list[str]` alertas	Arma el `.md` completo e imprime las alertas
`parse_args`, `main`	línea de comandos	código de salida	1 si hubo alguna alerta `REVISAR`

Librerías

OpenSSL (`<openssl/des.h>`, `-lcrypto`)

Función o tipo	Uso en el proyecto
`DES_cblock`	Bloque o llave de 8 bytes
`DES_key_schedule`	Subllaves de las 16 rondas
`DES_set_odd_parity`	Completa la paridad impar de cada byte de la llave
`DES_set_key_unchecked`	Genera el *key schedule* sin rechazar llaves débiles
`DES_ecb_encrypt`	Cifra o descifra un bloque de 8 bytes en modo ECB

Open MPI (`<mpi.h>`)

Primitiva	Uso en el proyecto
`MPI_Init`, `MPI_Finalize`	Abren y cierran MPI (`mpi_search_setup` y `mpi_search_release`)
`MPI_Init_thread`	El híbrido inicia MPI pidiendo `MPI_THREAD_FUNNELED`

Primitiva	Uso en el proyecto
`MPI_Initialized`	`mpi_search_setup` no vuelve a iniciar MPI si el híbrido ya lo hizo
`MPI_Comm_rank`, `MPI_Comm_size`	Rango propio y número de procesos
`MPI_Bcast`	Reparte el encabezado, el cifrado y la frase desde el proceso 0
`MPI_Barrier`	Sincroniza el inicio de la búsqueda
`MPI_Wtime`	Reloj de pared para T_p y para el límite de tiempo
`MPI_Irecv`	Antena de aviso y pedido adelantado del maestro-trabajador
`MPI_Send`	Aviso de llave, pedidos y respuestas del maestro-trabajador
`MPI_Recv`	El maestro recibe pedidos; el trabajador recibe chunks y STOP
`MPI_Test`	Sondeo de la antena cada 4096 llaves (hace avanzar la recepción)
`MPI_Wait`	Cierra la antena y espera el chunk adelantado
`MPI_Reduce`	Suma de las llaves probadas por todos
`MPI_Abort`	Solo si falta memoria en un proceso

libc

Función	Uso en el proyecto
`memcpy`	Copia del cifrado al `workspace` en cada intento
`memset`, `memchr`	Inicializar estructuras; detectar bytes nulos en el `.txt`
`strstr`	Buscar la frase clave en el texto descifrado
`strlen`, `strcmp`, `strcat`, `strdup`	Frase, presets de llave, optstring y copia de la frase
`getopt`	Leer las opciones de la línea de comandos
`strtoull`	Convertir números de 64 bits (`parse_u64`)
`malloc`, `calloc`, `free`	Memoria del problema y del `workspace` (`calloc` deja el relleno en ceros)
`fopen`, `fread`, `fwrite`, `fclose`, `ferror`	Leer el `.txt` y guardar el cifrado
`printf`, `fprintf`, `snprintf`, `fflush`, `perror`, `strerror`	Reportes y mensajes de error
`clock_gettime`	Reloj monotónico del secuencial
`nanosleep`	Pausa de 1 ms del proceso 0 mientras vigila el límite

OpenMP y C11 (híbrido)

Primitiva	Uso en el proyecto
`#pragma omp parallel num_threads(T) reduction(+ : iterations)`	Región paralela con T hilos; suma las llaves probadas de todos los hilos

Primitiva	Uso en el proyecto
`#pragma omp critical(found_key)`	Guarda llave e iteraciones del primer hilo que la halla
`omp_get_max_threads`	El proceso 0 lee T de `OMP_NUM_THREADS`
`omp_get_thread_num`	Número de hilo t, para el trabajador global g = r·T + t
`atomic_int`, `atomic_init`, `atomic_store`, `atomic_load_explicit` (`<stdatomic.h>`)	Bandera de parada compartida entre los hilos de un proceso

Python (análisis)

Módulo	Uso
`csv`, `statistics.median`	Leer la campaña y tomar la mediana de las repeticiones
`math.log2`	Bits extra según Gustafson
`argparse`, `os`, `re`, `sys`	Línea de comandos, rutas y alertas
`matplotlib` (opcional)	Gráficas en `docs/img/`

Anexo 2 · Bitácora de pruebas y speedups

Máquina y software

Mediciones **preliminares** en la laptop de Esteban (la campaña final se repite en la MacBook Pro de Nico con el mismo procedimiento):

CPU	Intel Core i9-13980HX: 24 núcleos (8 de rendimiento con *hyperthreading* + 16 de eficiencia), 32 hilos
Memoria	16 GB
Sistema	CachyOS (Linux 7.2.9)
Compilador	GCC 16.2.1 con `-O2 -march=native -std=c11` (los mismos en todos los programas)
MPI y librerías	Open MPI 5.0.11 · OpenSSL 3.6.5 · OpenMP de GCC

La ficha completa (`lscpu` y versión de `mpirun`) queda en `resultados/maquina_escu.txt`.

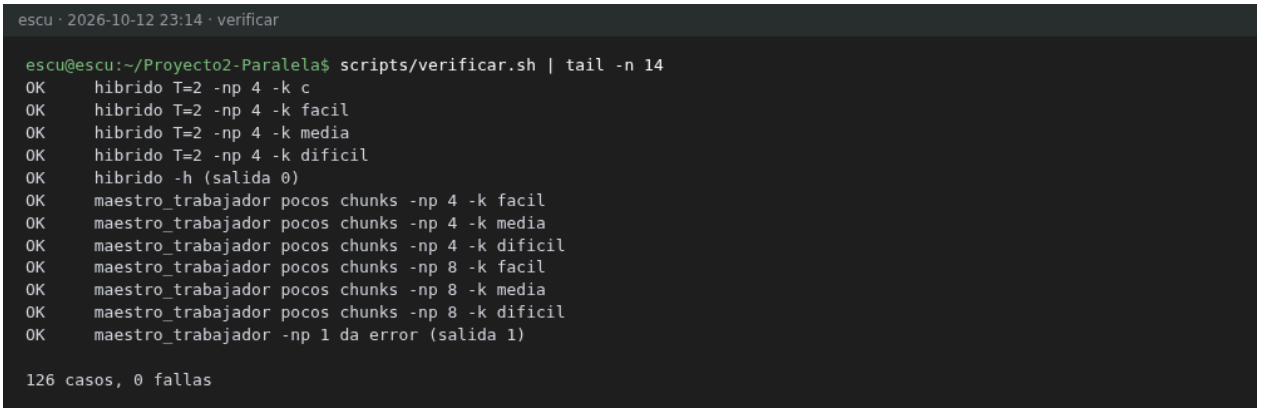
Procedimiento

```
make clean && make
scripts/verificar.sh                                  # 126 casos, 0 fallas
BITS=28 REPS=5 NP=4 NP_LIST="2 4 8" HYBRID_THREADS="1 2 4" scripts/campana.sh
python3 scripts/analizar.py resultados/campana_escu_<fecha>.csv
python3 scripts/capturas.py                           # capturas de este anexo
```

- **Bits:** 28 (N = 2^28 = 268 435 456). Los presets escalan con N, así que cada llave conserva la posición relativa que tiene con 56 bits.
- **Llaves:** fácil (N/2 + 1 = 134 217 729), media (N/2 + N/8 = 167 772 160) y difícil (⌈N/7 + N/13⌉ = 58 996 804).
- **Repeticiones:** 5 por caso; se reporta la **mediana** (y el mínimo y el máximo).
- **Tiempo:** de pared. En los programas MPI, `MPI_Wtime` desde la barrera hasta que llega el aviso; en el secuencial, `clock_gettime(CLOCK_MONOTONIC)` alrededor de la búsqueda.
- **T_s:** la mediana del programa `secuencial` con la misma llave y los mismos bits, no el paralelo con un proceso.
- **Híbrido:** `mpirun --bind-to none -x OMP_NUM_THREADS -np 4` con 1, 2 y 4 hilos.

Verificación

`scripts/verificar.sh` con `-b 20` corre cada programa con las 6 llaves (a, b, c, fácil, media y difícil) y `-np` 2, 3 y 4; el híbrido además con 1 y 2 hilos, y el maestro-trabajador con pocos chunks (`-b 18`, que agota los chunks antes de hallar la llave) y con `-np 1` (debe dar error sin colgarse). Resultado: **126 casos, 0 fallas**.



Final de `scripts/verificar.sh`: los últimos casos y el resumen.

Pruebas B.1 y B.2 (Nico)

Las corridas del cifrado (llave 42) y de las pruebas a, b y c del naive con `-np 4` están en el capítulo de pruebas B, con sus capturas y `resultados/pruebas_b2.csv`. En resumen: a da S ≈ 0.9 (mediana de 5), b no termina con 56 bits (≈ 149 años en secuencial; con 28 bits, S = 0.96) y c la halla el proceso 1 al primer intento (S aparente ≈ 3.9·10^12 con 56 bits).

Campaña preliminar: tiempos (mediana de 5, en segundos)

140 corridas; las 140 hallaron la llave real. Mínimo y máximo de cada caso en `resultados/analisis_escu.md`.

Programa	p	h	fácil	media	difícil
secuencial (T_s)	1	1	44.792	56.569	20.127
naive	4	1	0.004	14.326	25.321
intercalado	4	1	14.040	17.414	6.136
ciclico (C = 2^16)	4	1	13.942	17.645	6.152
maestro_trabajador (C = 2^16)	4	1	19.472	24.362	8.563

Programa	p	h	fácil	media	difícil
hibrido	4	1	13.499	16.783	6.009
hibrido	4	2	7.868	9.842	3.404
hibrido	4	4	4.044	5.801	2.029

Speedups (S = T_s / T_p)

Programa	w	fácil	media	difícil	E (media)
naive	4	11 087 (aparente)	3.95	0.79	0.99
intercalado	4	3.19	3.25	3.28	0.81
ciclico	4	3.21	3.21	3.27	0.80
maestro_trabajador	4	2.30	2.32	2.35	0.58
hibrido, 1 hilo	4	3.32	3.37	3.35	0.84
hibrido, 2 hilos	8	5.69	5.75	5.91	0.72
hibrido, 4 hilos	16	11.08	9.75	9.92	0.61

Alertas del análisis: ninguna. Ningún enfoque alternativo da S > w y ningún programa da S_rend > w.

Barrido de procesos (llave media)

Programa	p = 2	p = 4	p = 8
intercalado: T_p (s) / S	32.565 / 1.74	17.414 / 3.25	11.301 / 5.01
maestro_trabajador: T_p (s) / S	67.133 / 0.84	24.362 / 2.32	13.477 / 4.20

Capturas

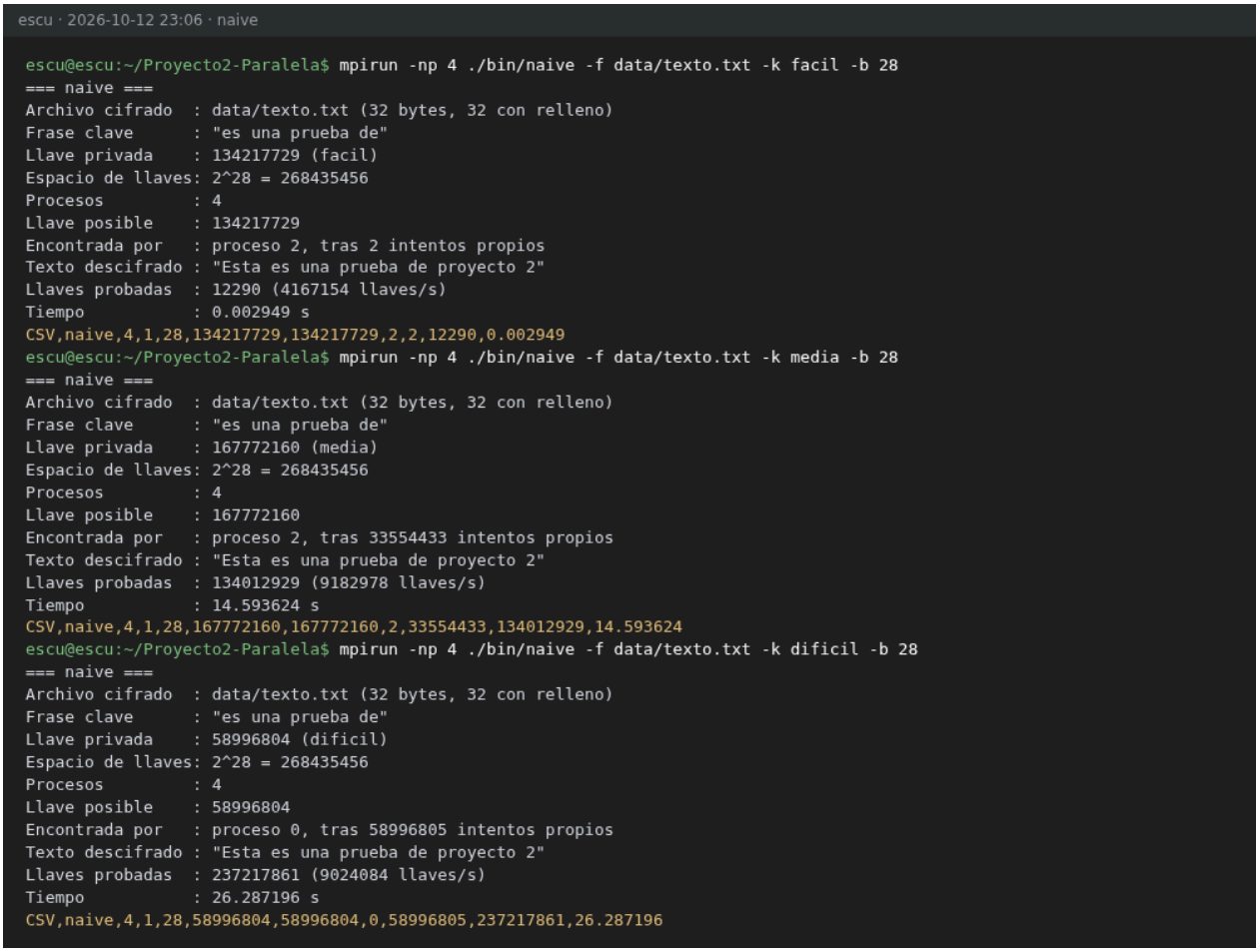
Cada captura es la salida real de los comandos, tal como la imprimen los programas, tomada con `scripts/capturas.py` en la misma máquina después de la campaña (los tiempos de una corrida suelta varían unas décimas respecto a la mediana).

Secuencial (T_s)

```
escu · 2026-10-12 23:05 · secuencial

escu@escu:~/Proyecto2-Paralela$ ./bin/secuencial -f data/texto.txt -k facil -b 28
=== secuencial ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 134217729 (facil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 1
Llave posible    : 134217729
Encontrada por   : proceso 0, tras 134217730 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 134217730 (3010024 llaves/s)
Tiempo           : 44.590246 s
CSV,secuencial,1,1,28,134217729,134217729,0,134217730,134217730,44.590246
escu@escu:~/Proyecto2-Paralela$ ./bin/secuencial -f data/texto.txt -k media -b 28
=== secuencial ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 167772160 (media)
Espacio de llaves: 2^28 = 268435456
Procesos         : 1
Llave posible    : 167772160
Encontrada por   : proceso 0, tras 167772161 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 167772161 (3035485 llaves/s)
Tiempo           : 55.270306 s
CSV,secuencial,1,1,28,167772160,167772160,0,167772161,167772161,55.270306
escu@escu:~/Proyecto2-Paralela$ ./bin/secuencial -f data/texto.txt -k dificil -b 28
=== secuencial ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 58996804 (dificil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 1
Llave posible    : 58996804
Encontrada por   : proceso 0, tras 58996805 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 58996805 (3053659 llaves/s)
Tiempo           : 19.320039 s
CSV,secuencial,1,1,28,58996804,58996804,0,58996805,58996805,19.320039
```

Naive



Fácil: el proceso 2 la halla al segundo intento. Media: el proceso 2 a la mitad de su bloque. Difícil: el proceso 0, con las mismas iteraciones que el secuencial.

Intercalado

```
escu · 2026-10-12 23:07 · intercalado

escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/intercalado -f data/texto.txt -k facil -b 28
=== intercalado ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 134217729 (facil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 134217729
Encontrada por   : proceso 1, tras 33554433 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 134008833 (9401604 llaves/s)
Tiempo           : 14.253826 s
CSV,intercalado,4,1,28,134217729,134217729,1,33554433,134008833,14.253826
escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/intercalado -f data/texto.txt -k media -b 28
=== intercalado ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 167772160 (media)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 167772160
Encontrada por   : proceso 0, tras 41943041 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 168636417 (8888012 llaves/s)
Tiempo           : 18.973469 s
CSV,intercalado,4,1,28,167772160,167772160,0,41943041,168636417,18.973469
escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/intercalado -f data/texto.txt -k dificil -b 28
=== intercalado ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 58996804 (dificil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 58996804
Encontrada por   : proceso 0, tras 14749202 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 59354642 (8816712 llaves/s)
Tiempo           : 6.732061 s
CSV,intercalado,4,1,28,58996804,58996804,0,14749202,59354642,6.732061
```

Bloque-cíclico

```
escu · 2026-10-12 23:08 · ciclico

escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/ciclico -f data/texto.txt -k facil -b 28
=== ciclico ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 134217729 (facil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 134217729
Encontrada por   : proceso 0, tras 33554434 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 134889474 (9285492 llaves/s)
Tiempo           : 14.526906 s
CSV,ciclico,4,1,28,134217729,134217729,0,33554434,134889474,14.526906
escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/ciclico -f data/texto.txt -k media -b 28
=== ciclico ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 167772160 (media)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 167772160
Encontrada por   : proceso 0, tras 41943041 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 168652801 (9436484 llaves/s)
Tiempo           : 17.872420 s
CSV,ciclico,4,1,28,167772160,167772160,0,41943041,168652801,17.872420
escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/ciclico -f data/texto.txt -k dificil -b 28
=== ciclico ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 58996804 (dificil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 58996804
Encontrada por   : proceso 0, tras 14760005 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 59344965 (9569131 llaves/s)
Tiempo           : 6.201709 s
CSV,ciclico,4,1,28,58996804,58996804,0,14760005,59344965,6.201709
```

Maestro-trabajador

```
escu · 2026-10-12 23:09 · maestro_trabajador

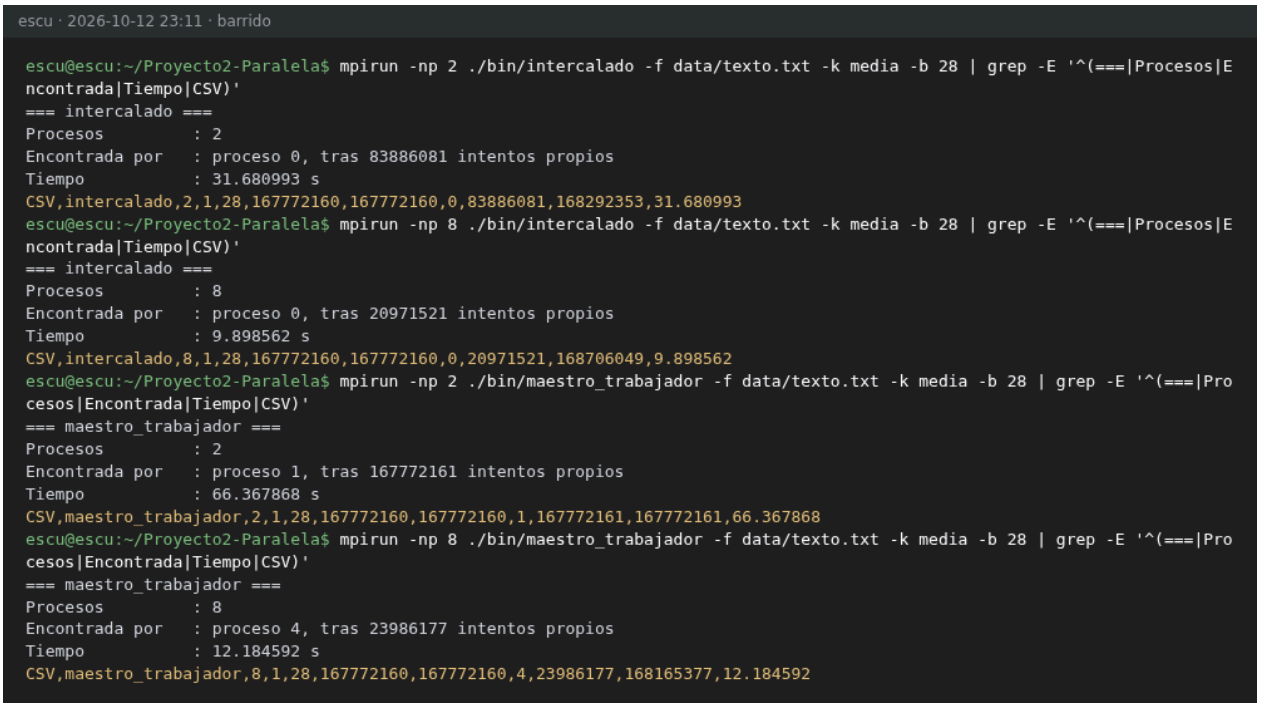
escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/maestro_trabajador -f data/texto.txt -k facil -b 28
=== maestro_trabajador ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 134217729 (facil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 134217729
Encontrada por   : proceso 2, tras 44695554 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 134348802 (6851069 llaves/s)
Tiempo           : 19.609904 s
CSV,maestro_trabajador,4,1,28,134217729,134217729,2,44695554,134348802,19.609904
escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/maestro_trabajador -f data/texto.txt -k media -b 28
=== maestro_trabajador ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 167772160 (media)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 167772160
Encontrada por   : proceso 3, tras 55902209 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 167903233 (6852056 llaves/s)
Tiempo           : 24.504064 s
CSV,maestro_trabajador,4,1,28,167772160,167772160,3,55902209,167903233,24.504064
escu@escu:~/Proyecto2-Paralela$ mpirun -np 4 ./bin/maestro_trabajador -f data/texto.txt -k dificil -b 28
=== maestro_trabajador ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 58996804 (dificil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Llave posible    : 58996804
Encontrada por   : proceso 2, tras 19675205 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 59258949 (6875113 llaves/s)
Tiempo           : 8.619341 s
CSV,maestro_trabajador,4,1,28,58996804,58996804,2,19675205,59258949,8.619341
```

Híbrido (4 procesos × 4 hilos)

```
escu · 2026-10-12 23:09 · hibrido

escu@escu:~/Proyecto2-Paralela$ OMP_NUM_THREADS=4 mpirun --bind-to none -x OMP_NUM_THREADS -np 4 ./bin/hibrido -f data/text
o.txt -k facil -b 28
=== hibrido ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 134217729 (facil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Hilos por proceso: 4 (16 trabajadores)
Llave posible    : 134217729
Encontrada por   : proceso 0, tras 8388609 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 119779329 (30314808 llaves/s)
Tiempo           : 3.951182 s
CSV,hibrido,4,4,28,134217729,134217729,0,8388609,119779329,3.951182
escu@escu:~/Proyecto2-Paralela$ OMP_NUM_THREADS=4 mpirun --bind-to none -x OMP_NUM_THREADS -np 4 ./bin/hibrido -f data/text
o.txt -k media -b 28
=== hibrido ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 167772160 (media)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Hilos por proceso: 4 (16 trabajadores)
Llave posible    : 167772160
Encontrada por   : proceso 0, tras 10485761 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 169598977 (29706625 llaves/s)
Tiempo           : 5.709130 s
CSV,hibrido,4,4,28,167772160,167772160,0,10485761,169598977,5.709130
escu@escu:~/Proyecto2-Paralela$ OMP_NUM_THREADS=4 mpirun --bind-to none -x OMP_NUM_THREADS -np 4 ./bin/hibrido -f data/text
o.txt -k dificil -b 28
=== hibrido ===
Archivo cifrado  : data/texto.txt (32 bytes, 32 con relleno)
Frase clave      : "es una prueba de"
Llave privada    : 58996804 (dificil)
Espacio de llaves: 2^28 = 268435456
Procesos         : 4
Hilos por proceso: 4 (16 trabajadores)
Llave posible    : 58996804
Encontrada por   : proceso 1, tras 3687301 intentos propios
Texto descifrado : "Esta es una prueba de proyecto 2"
Llaves probadas  : 58053509 (29387359 llaves/s)
Tiempo           : 1.975459 s
CSV,hibrido,4,4,28,58996804,58996804,1,3687301,58053509,1.975459
```

Barrido de procesos



Intercalado y maestro-trabajador con 2 y 8 procesos (se muestran solo las líneas clave de cada reporte).